CS530 Project Report: Auto Drone Simulation Navigation

Team Members: Yupu Guo, Jingdi Wu, Sooyoung Kim
Github repo: https://github.com/Colossus-MKII/cs530-project

1. Abstract

This report presents the design and implementation of an autonomous drone navigation system in the Unreal Engine-based AirSimNH environment. The project explores both classical planning-based navigation and learning-based control. First, we implemented a hybrid classical pipeline that combines A* global planning, Artificial Potential Field (APF) local obstacle avoidance, and a finite-state recovery mechanism to reduce deadlock behavior caused by local minima. The system uses a top-down map extracted from AirSim and converts it into a grid representation for path planning, while LiDAR-based local sensing supports reactive obstacle avoidance.

In addition to the classical pipeline, we developed a Deep Q-Network (DQN)-based reinforcement learning navigation agent for 3D drone control. The RL system uses LiDAR-derived obstacle features, relative goal information, altitude state, and local safety indicators to learn navigation policies through interaction with the AirSim environment. A major focus of this work is reward engineering. We designed a dense multi-objective reward function that combines 3D goal progress, obstacle-distance penalties, altitude regulation, ROI boundary constraints, collision termination, and goal completion reward. We also compared soft and hard boundary strategies and found that hard boundary termination produced more stable learning behavior and more focused exploration in the constrained training region.

Overall, the project demonstrates that classical planning methods provide reliable global structure, while reinforcement learning offers a flexible framework for adaptive local navigation. The experiments show that reward shaping, altitude control, LiDAR-based safety penalties, and boundary handling are critical for training a usable drone navigation policy in a complex 3D simulation environment.

2. Environment Setup

AirSim is an open-source simulator for drones, cars and more, built on Unreal Engine. You can download its latest version (v1.8.1) from github: https://github.com/microsoft/airsim/releases

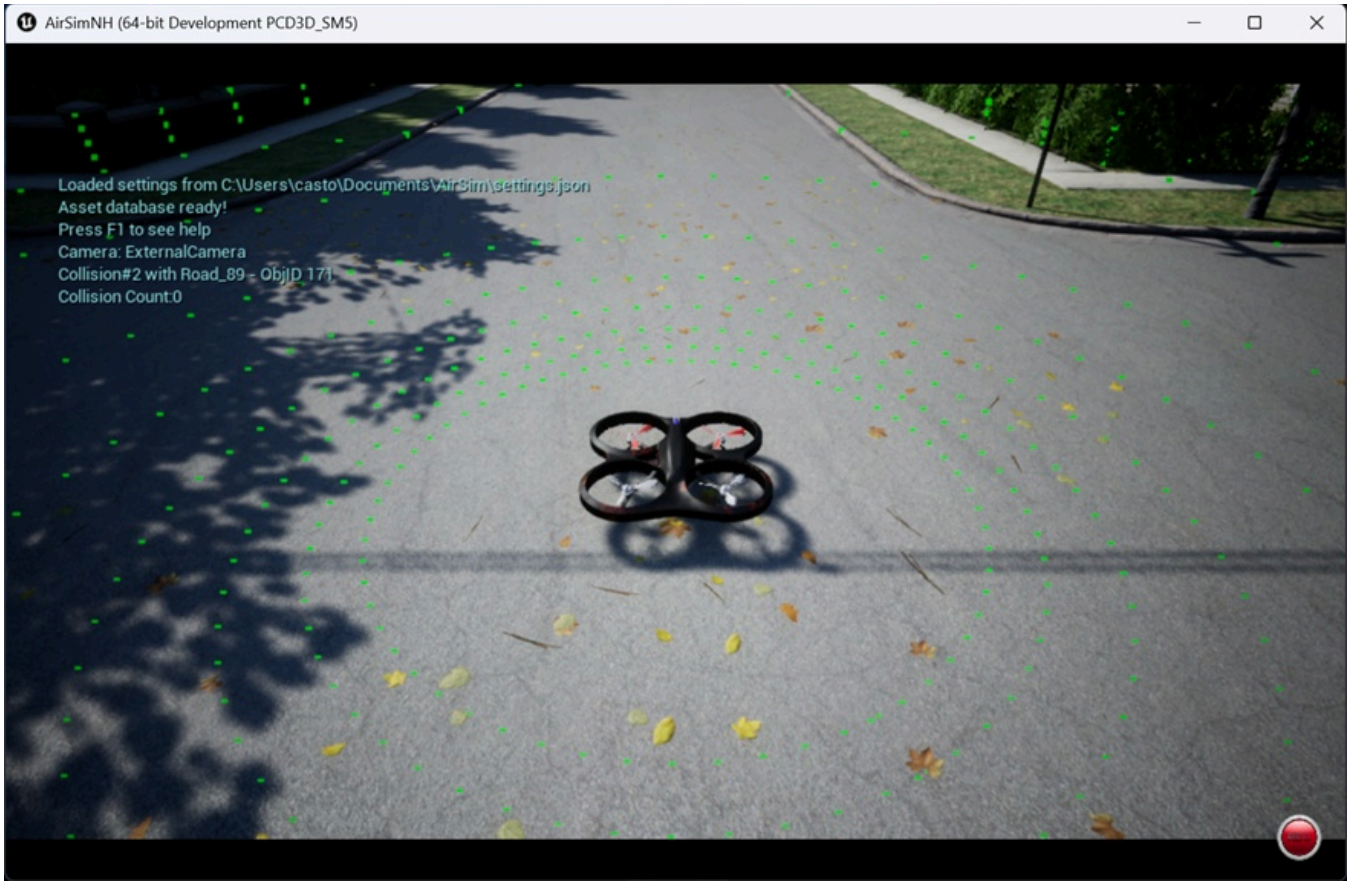


Figure 1: AirSim Environment

AirSim exposes APIs so we can interact with the drone in the simulation programmatically. You can use these APIs to retrieve images, get state, control the vehicle and so on. The APIs are exposed through the RPC, and are accessible via a variety of languages, including C++, Python, C# and Java.
We choose a town map AirSimNH as our experiment environment (Around 250m × 250m).

2.1 Fetch native 2D top-down map and grid map required for pathfinding.

The challenge is AirSim doesn't provide the native topography. So we flied the drone to 200m height to capture a high-resolution (1024 * 1024). Then we processed the image to create `astar_grid.npy` to discretizing obstacles for algorithm use.

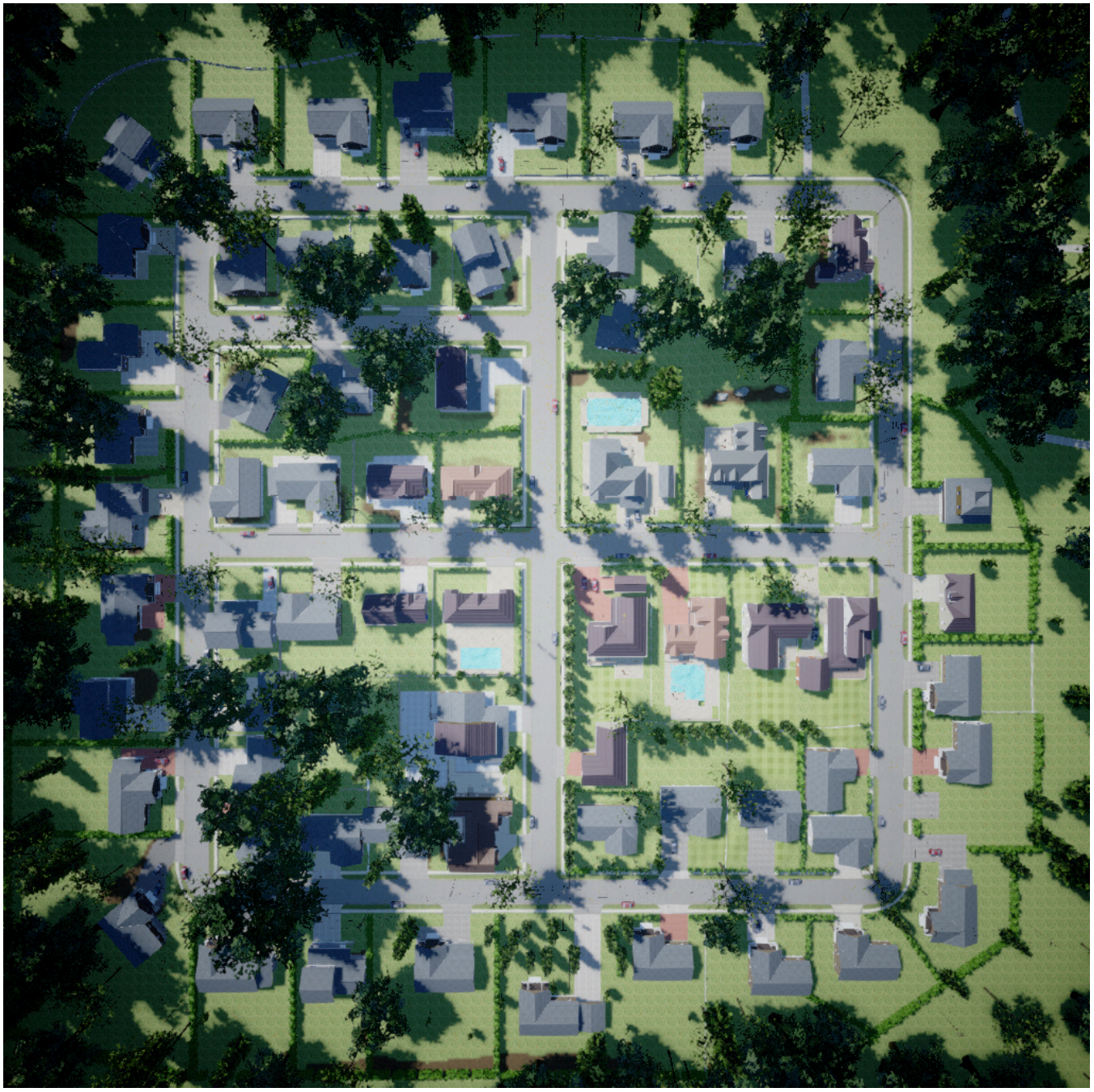


Figure 2: 2D top-down satellite map

2.2 Create the global delivery map

Then we use `telemetry_map.py` to set up an application from former satellite map as following. In this application, user can fetch the position, altitude, speed information, the boundary of the map, and trace teh drone's flight path.

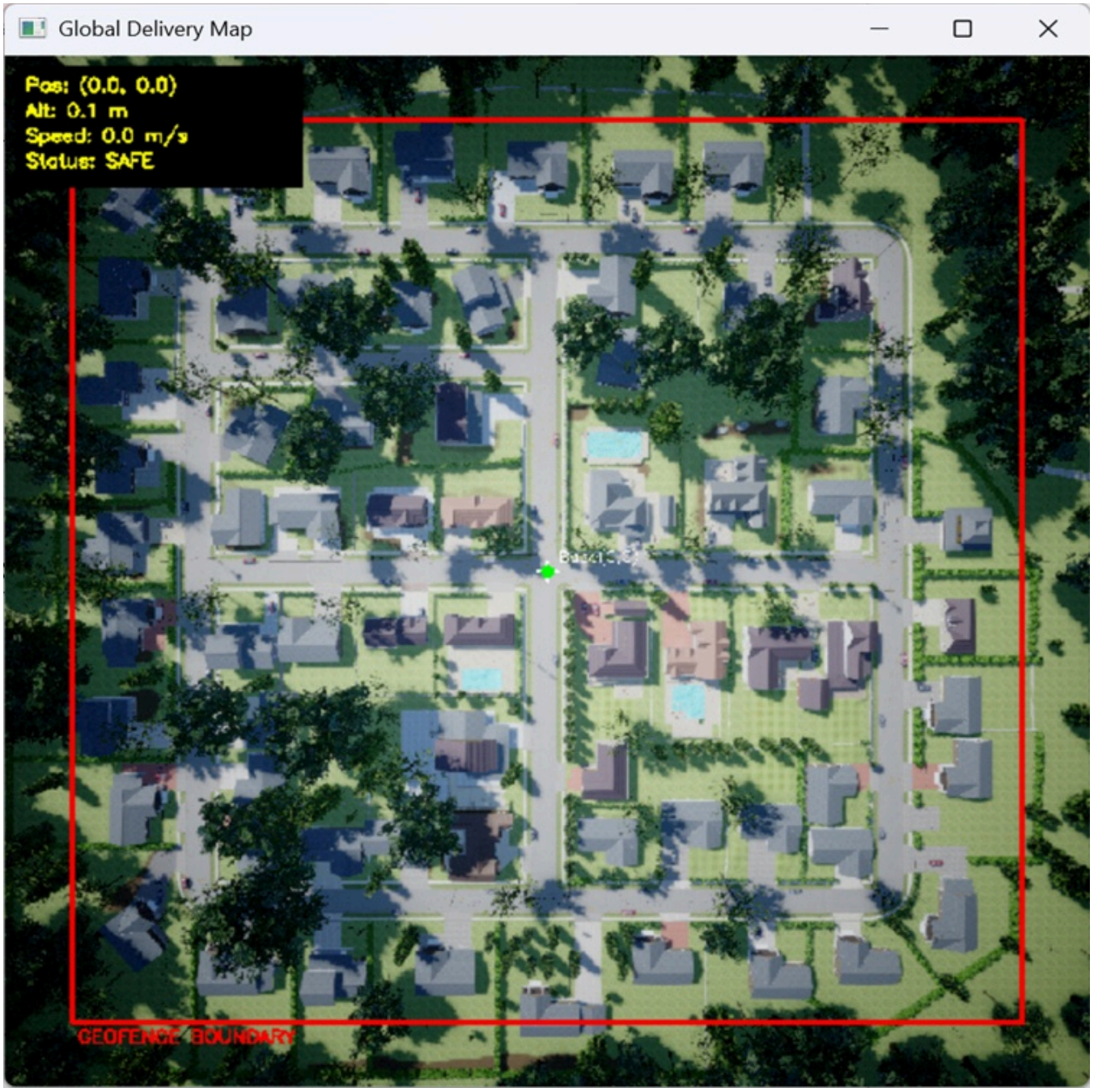


Figure 3: Global Delivery Map

2.3 Setup Interactive Interface

And we use the `autonomous_delivery.py` to set up a OpenCV-based GUI for user to choose the start and end. The application will use A* algorithm to plan a path and use Artificial Potential Field (APF) to avoid obstacles that isn't presented in 2D map.
The application also controls the drone's flight in *AirSimNH.exe*.

Figure 4: Hybrid Navigator

3. Algorithms

3.1 Global Planning with A*

For macro-level navigation across the static grid, we implemented the A* search algorithm. This acts as the brain of the system, calculating the absolute shortest path before takeoff.

The algorithm minimizes the total cost function $f(n)$:
$f(n) = g(n) + h(n)$

- **Actual Cost $g(n)$:** Represents the accumulated physical flight distance from the starting node to the current node n. To accurately model energy expenditure, orthogonal movements are assigned a cost of 1, while diagonal movements are assigned a cost of ≈ 1.414 ($\sqrt{2}$).

- **Heuristic Cost $h(n)$:** We utilized the Euclidean distance as the heuristic to guide the search greedily toward the destination:$h(n) = \sqrt{(x_n - x_{goal})^2 + (y_n - y_{goal})^2}$

This ensures optimal energy consumption over long distances in static environments but lacks the reactivity needed for dynamic or unmapped obstacles.

3.2 Local Reactive Avoidance: Artificial Potential Field (APF)

To compensate for A*'s rigidity, we implemented an APF controller that acts as the drone's "Cerebellum," operating at a high-frequency control loop (10Hz) for instantaneous evasive maneuvers.

3.2.1 Lidar-Driven Force Vectors

The drone's 360-degree Lidar point cloud is compressed into 8 horizontal sectors to reduce computational overhead. The flight velocity is dictated by the linear superposition of two virtual forces:

- **Attractive Force (Goal's Pull):** Generates a normalized velocity vector continuously pulling the drone toward the final destination.
- **Repulsive Force (Obstacle's Push):** If an obstacle is detected within the critical threshold (10m), a repulsive vector is generated. The magnitude of this force scales inversely with proximity: $\text{Magnitude} \propto (\text{Threshold} - \text{Current Distance})$
- **Vector Addition:** The final commanded velocity is the sum of these vectors ($V_{target} = V_{att} + V_{rep}$), enabling smooth, reflex-like evasive maneuvers without stopping.

3.3 FSM & Anti-Deadlock Recovery Mechanism

The most significant flaw in standard APF is the "Local Minima" trap, where attractive and repulsive forces cancel out, causing the drone to deadlock or oscillate infinitely. To solve this, we designed a Finite State Machine (FSM) with a **Heuristic Recovery Mechanism**.

3.3.1 The Hysteresis Threshold Logic

When the drone deviates from the A* path to avoid a threat, it records its current Euclidean distance to the goal (deviation_progress). After clearing the obstacle, the drone executes the following recovery protocol:

```
elif fsm_state == STATE_AVOID:
            # APF Baseline
            dist_g = math.hypot(last_x - pos.x_val, last_y - pos.y_val)
            att_x, att_y = (last_x-pos.x_val)/dist_g*MAX_SPEED, (last_y-
pos.y_val)/dist_g*MAX_SPEED
            rep_x, rep_y = 0.0, 0.0
            for i, d in enumerate(obs_sectors):
                if d < APF_TRIGGER_DIST:
                    ang = (i/8.0)*2*math.pi-math.pi
                    mag = (APF_TRIGGER_DIST - d) * 45.0
                    rep_x -= math.cos(ang)*mag
                    rep_y -= math.sin(ang)*mag
            vx, vy = att_x + rep_x, att_y + rep_y
```

After avoiding an obstacle, the drone scans the remaining A* route and snaps to a waypoint with a strictly smaller heuristic value (closer to the destination).

4. Reinforcement Learning-based Autonomous Navigation

4.1 RL Environment and State Representation

To enable adaptive autonomous navigation in complex suburban environments, we implemented a Deep Q-Network (DQN)-based reinforcement learning framework inside Microsoft AirSim. Unlike traditional global path planners that rely on static map assumptions, the RL agent directly learns navigation policies through continuous interaction with the environment.

The training environment contains:

- static buildings and vegetation obstacles,
- constrained 3D flight regions,
- long-range navigation objectives,
- partial observability through onboard sensing.

A rectangular Region of Interest (ROI) was introduced to limit the training area and reduce meaningless exploration outside the target operational space.

4.1.1 State Space

The agent state vector combines multiple sources of spatial and safety information:

- discretized LiDAR distance sectors,
- relative goal position,
- drone altitude,
- velocity-related information,
- local obstacle proximity.

This state representation allows the policy to jointly reason about:

- obstacle avoidance,
- directional movement,
- altitude stabilization,
- long-range goal progression.

Unlike traditional 2D navigation tasks, our environment requires simultaneous control in both horizontal and vertical dimensions, significantly increasing policy complexity.

4.1.2 Action Space

We adopted a discrete action space to improve DQN training stability. The available actions include:

- forward movement,
- yaw-left rotation,
- yaw-right rotation,
- ascend,
- descend,

- hover stabilization.

Although continuous control methods such as PPO or SAC may provide smoother motion, discrete DQN training was substantially easier to stabilize during early experimentation.

4.2 Reward Engineering

Reward engineering became the most critical component of the entire RL system. Initial sparse-reward experiments using only terminal goal rewards produced unstable exploration behavior and extremely slow convergence. The drone frequently wandered without meaningful directional learning.

To address this issue, we designed a dense multi-objective reward system that simultaneously optimizes:

- navigation efficiency,
- obstacle safety,
- altitude regulation,
- ROI compliance,
- collision avoidance.

4.2.1 Motivation for Reward Shaping

A naive sparse reward formulation:

```
+100 when reaching goal
```

was insufficient for large-scale 3D environments because the probability of randomly reaching the destination during early exploration was extremely low.

As a result:

- the replay buffer became dominated by meaningless transitions,
- the policy failed to learn directional movement,
- exploration became highly unstable.

Therefore, we introduced continuous reward shaping terms that provide immediate feedback during navigation.

4.2.2 Progress-based Navigation Reward

The core navigation signal is based on 3D Euclidean progress toward the goal:

```
progress = prev_distance_to_goal - current_distance
```

The reward formulation is:

```
progress_reward = 5.0 * progress
```

A small per-step penalty was also introduced:

```
step_penalty = -0.2
```

This design encourages:

- continuous movement toward the destination,
- shorter trajectories,
- reduced hovering behavior.

Unlike sparse rewards, this dense formulation continuously guides the agent during long-distance exploration.

4.2.3 Obstacle-aware Safety Penalty

To improve safety behavior, we designed a multi-level LiDAR penalty mechanism based on the minimum detected obstacle distance.

Instead of penalizing only direct collisions, the agent receives progressively stronger penalties when approaching obstacles:

```
if min_lidar < 0.8:
    obstacle_penalty = -20.0
elif min_lidar < 1.5:
    obstacle_penalty = -5.0
elif min_lidar < 2.5:
    obstacle_penalty = -1.0
```

This hierarchical penalty structure enables:

- proactive obstacle avoidance,
- safer local maneuvering,
- smoother trajectory generation.

The policy therefore learns not only to avoid collisions, but also to maintain safe obstacle margins during flight.

4.2.4 Goal Reward

A large terminal reward is assigned when the drone reaches the target region:

```
if current_distance < 1.0:
    goal_reward = 100.0
```

This reward defines the primary optimization objective of the navigation task.

Once the goal is reached, the episode terminates immediately.

4.2.5 Altitude Regulation

Unlike many 2D RL navigation tasks, our environment requires stable 3D flight control.

To prevent unrealistic altitude behavior, we introduced altitude-related constraints and energy penalties.

Altitude Energy Cost

```
altitude_energy_cost = -0.02 * altitude
```

This term discourages unnecessary high-altitude flight and loosely approximates energy consumption.

Altitude Boundary Constraints

The drone must remain within:

- minimum altitude: 2m
- maximum altitude: 20m

Exceeding the maximum altitude triggers strong penalties:

```
too_high_penalty = -20.0 - 2.0 * excess_altitude
```

Flying below the minimum altitude immediately terminates the episode:

```
too_low_penalty = -50.0
```

This mechanism stabilizes vertical behavior and prevents unsafe flight trajectories.

4.2.6 ROI Boundary Constraint

To prevent uncontrolled exploration outside the training area, we introduced ROI boundary penalties.

Two different strategies were explored during experimentation:

Soft Boundary Strategy

The soft strategy only applies negative penalties when leaving the ROI while allowing the episode to continue.

This encourages exploration flexibility but often results in:

- wandering behavior,
- unstable navigation,
- inefficient exploration.

Hard Boundary Strategy

The hard strategy immediately terminates the episode once the drone exits the ROI.

This significantly improves:

- training efficiency,
- trajectory focus,
- policy convergence speed.

Importantly, the hard constraint does not eliminate failures entirely. Instead, it reshapes the failure distribution into more meaningful task-oriented exploration patterns.

4.2.7 Collision Termination

Real collisions are detected using AirSim collision feedback combined with custom filtering logic.

Once a collision occurs:

```
collision_penalty = -100.0
```

The episode terminates immediately.

This terminal constraint strongly reinforces safe navigation behavior.

4.3 Training Stability Analysis

To evaluate policy learning behavior, we analyzed reward convergence, success-rate evolution, training loss, and safety-related metrics.

Before examining individual training curves, we summarize the overall performance of the hard-boundary and soft-boundary reward settings. The hard-boundary setting achieved a substantially higher goal-reaching rate, lower collision rate, and lower timeout rate, while the soft-boundary setting produced longer unstable trajectories and more frequent failures.

Boundary Strategy	Episodes	Goal Reached	Collision	Timeout	Out of ROI	Too Low	Too High Warning
Hard Boundary	500	61.0%	7.8%	15.0%	16.2%	N/A	8.0%
Soft Boundary	800	27.6%	20.4%	46.6%	N/A	5.4%	12.6%

Here, N/A means that the event was not used as a final terminal category in that specific training setting.

This comparison shows that the hard-boundary design is more suitable for the constrained AirSim training region. Although soft boundary penalties are more flexible, they allow the agent to accumulate many low-quality exploratory transitions, which makes DQN training less stable.

4.3.1 Reward Curve Analysis

Hard Reward Result

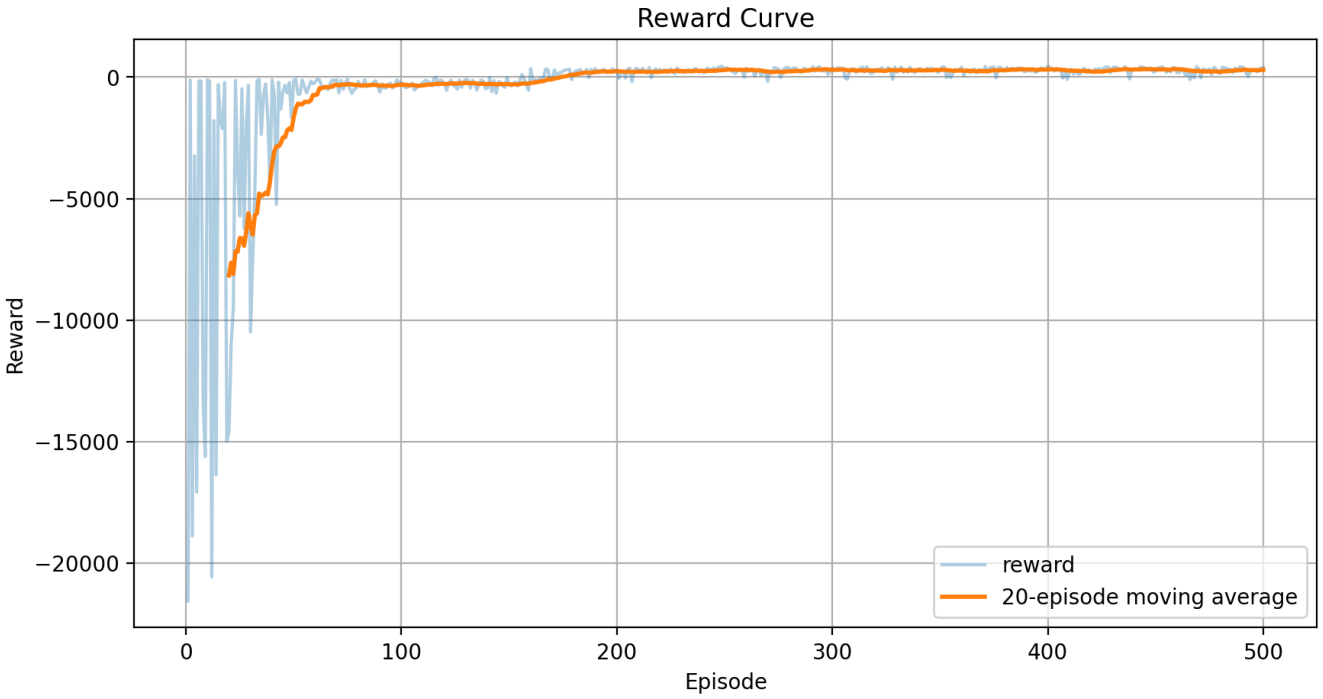


Figure 5: Reward curve under the hard-boundary strategy.

Soft Reward Result

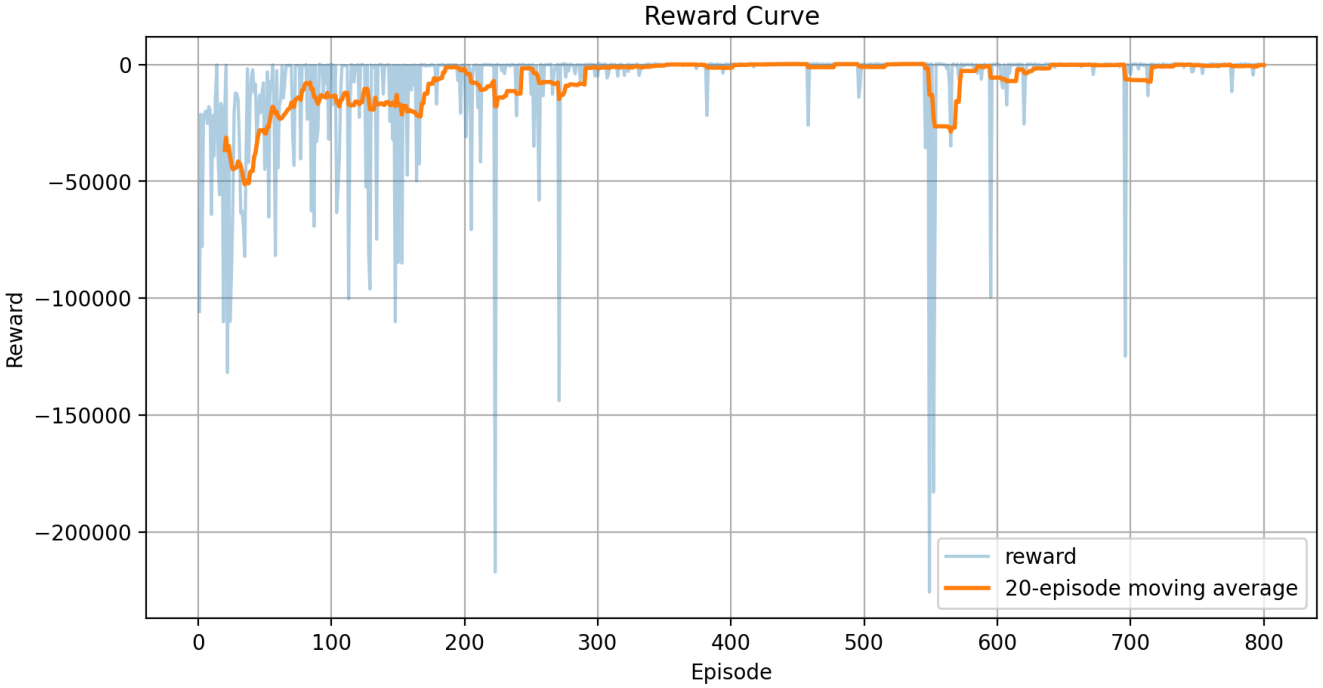


Figure 6: Reward curve under the soft-boundary strategy.

The hard-boundary reward design demonstrates faster convergence and significantly reduced oscillation during late-stage training.

In contrast, the soft-boundary formulation exhibits prolonged instability caused by excessive exploration outside the intended navigation region.

The results suggest that strict environmental constraints improve policy optimization efficiency in sparse-goal navigation tasks.

4.3.2 Success Rate Evolution

Hard Reward Result

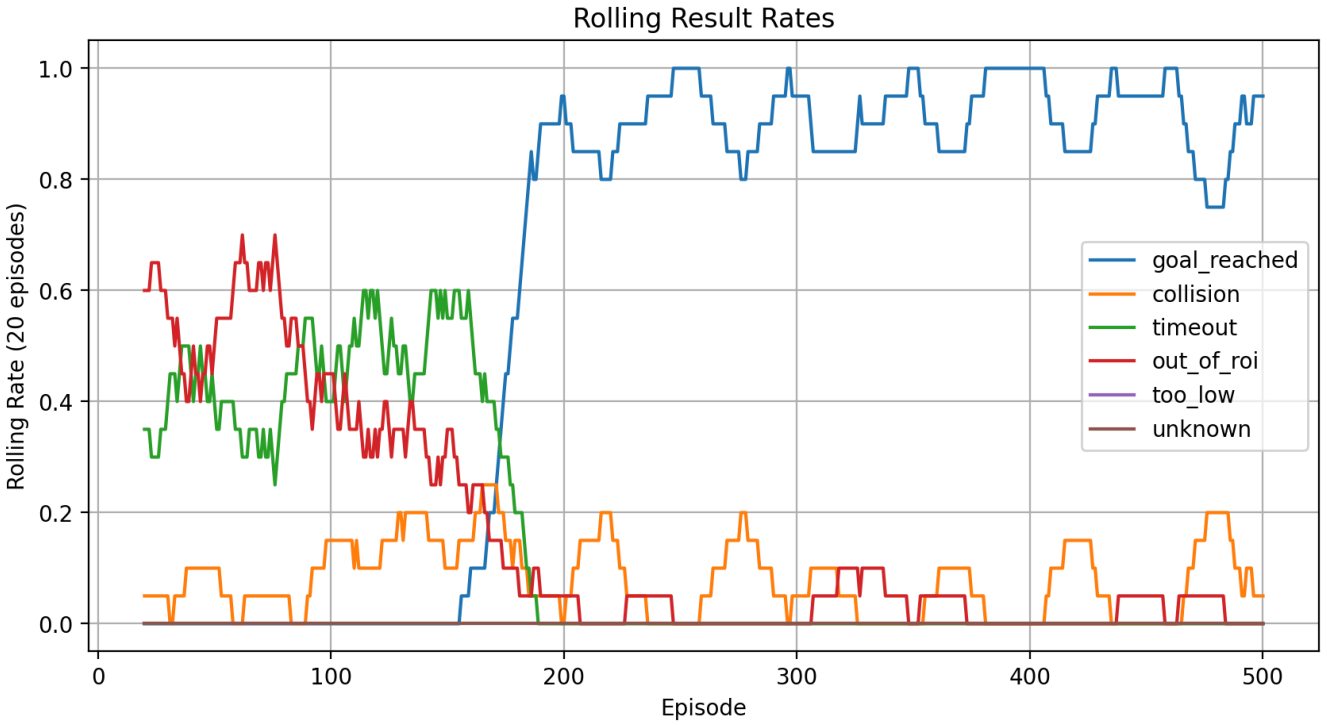


Figure 7: Rolling episode result rates under the hard-boundary strategy.

Soft Reward Result

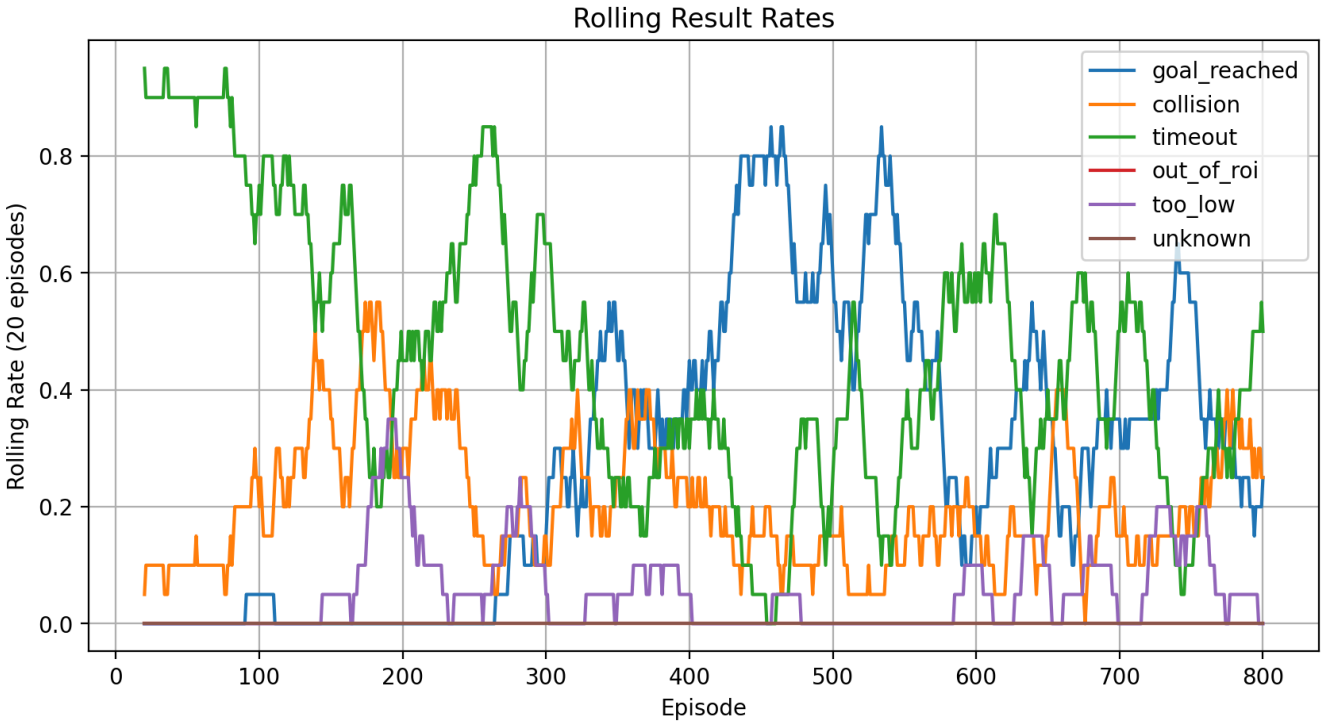


Figure 8: Rolling episode result rates under the soft-boundary strategy.

The rolling success-rate curves clearly demonstrate that the hard-boundary reward system achieves faster policy stabilization and improved navigation consistency.

The soft-boundary system requires substantially longer exploration before reaching comparable performance levels.

4.3.3 Loss Curve Analysis

Hard Reward Result

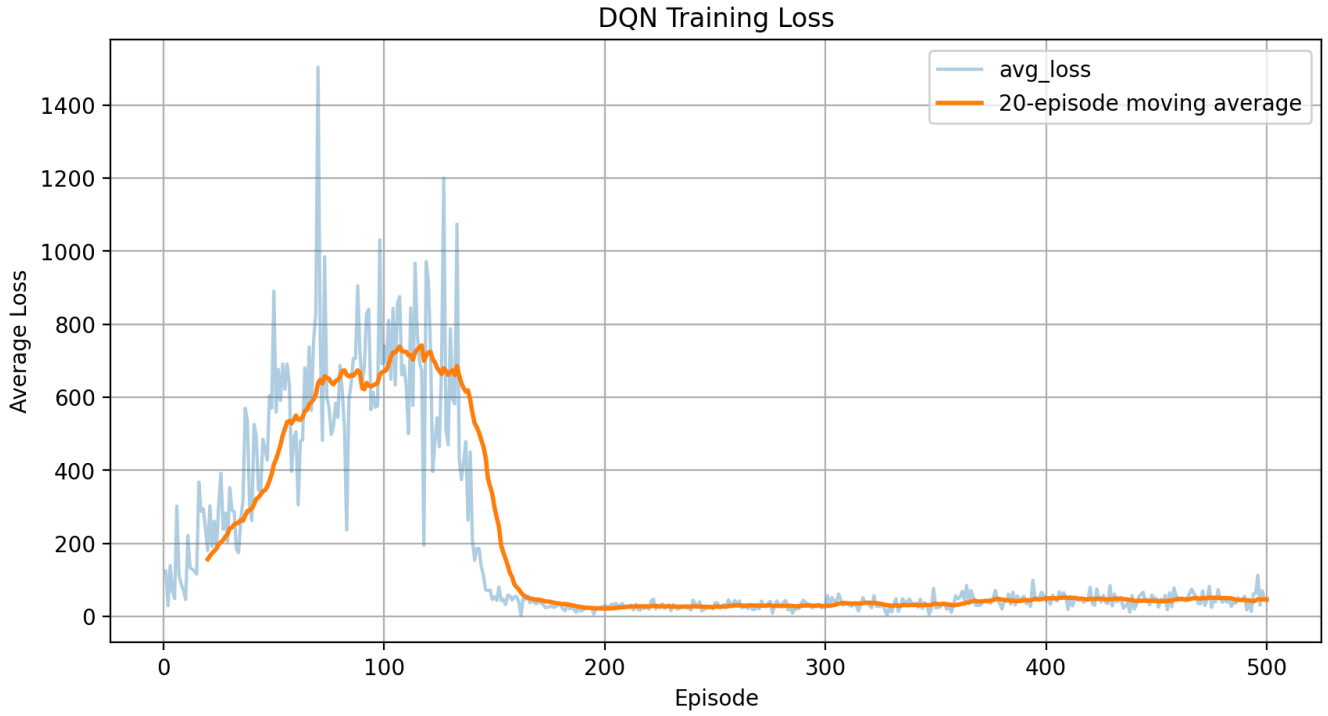


Figure 9: DQN loss curve under the hard-boundary strategy.

Soft Reward Result

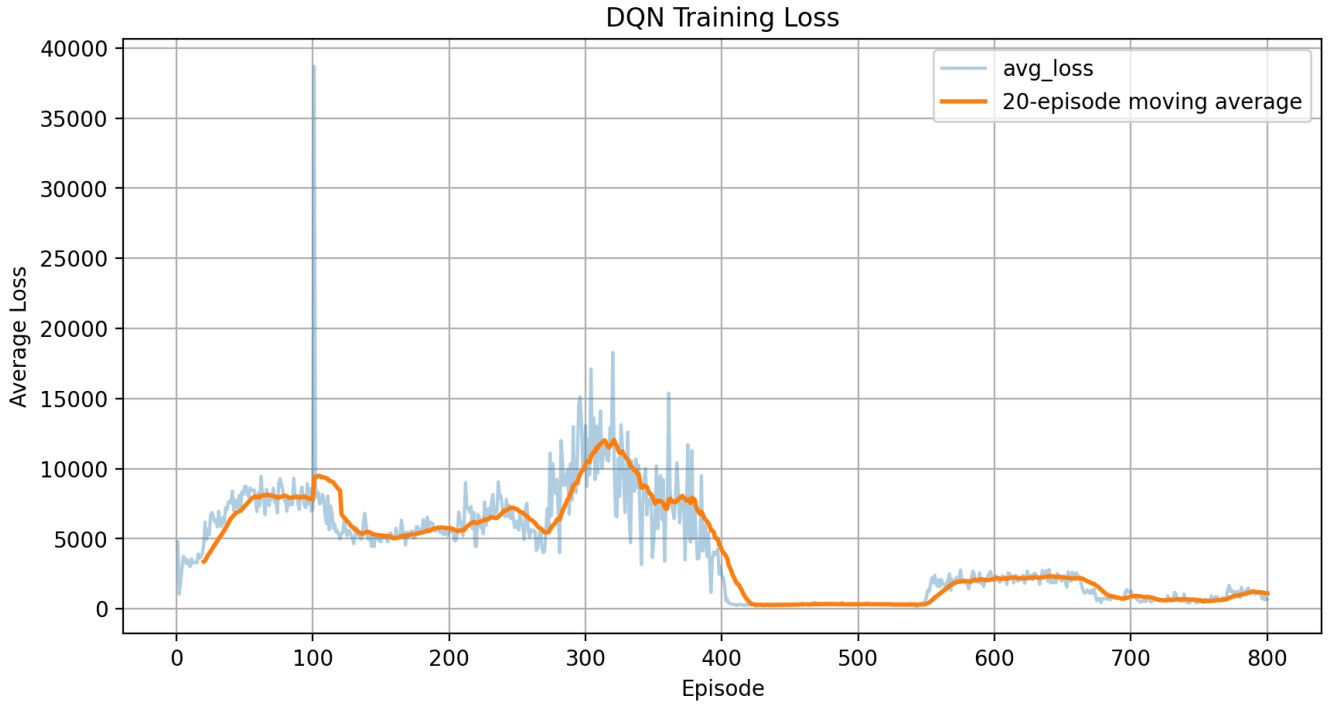


Figure 10: DQN loss curve under the soft-boundary strategy.

Although DQN training loss remains noisy due to stochastic replay sampling and exploration variance, both systems eventually converge toward relatively stable optimization behavior.

The hard-boundary design exhibits lower long-term variance, indicating more stable policy updates.

4.3.4 Safety and Altitude Metrics

Hard Reward Result

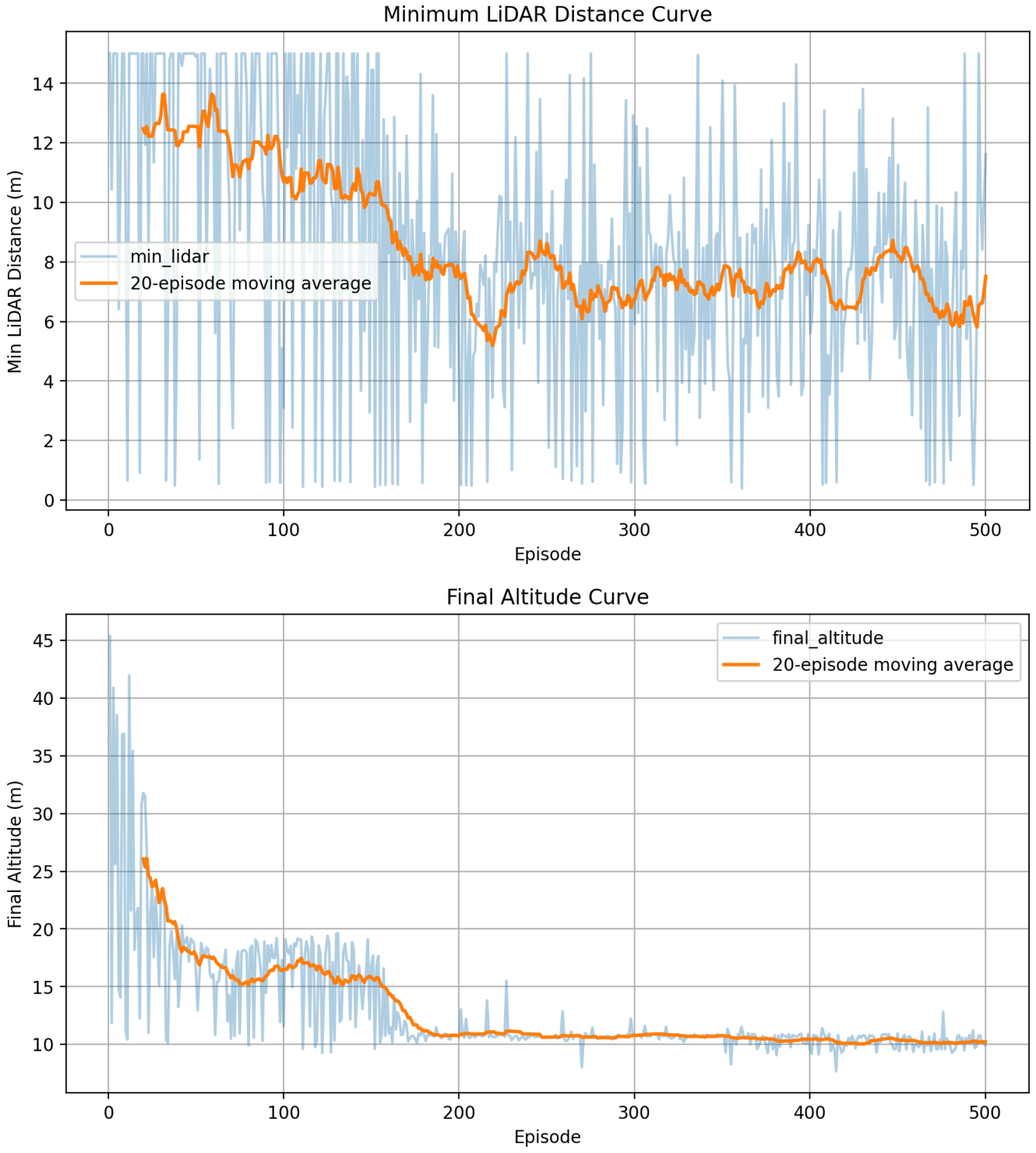


Figure 11: Safety and altitude metrics under the hard-boundary strategy.

Soft Reward Result

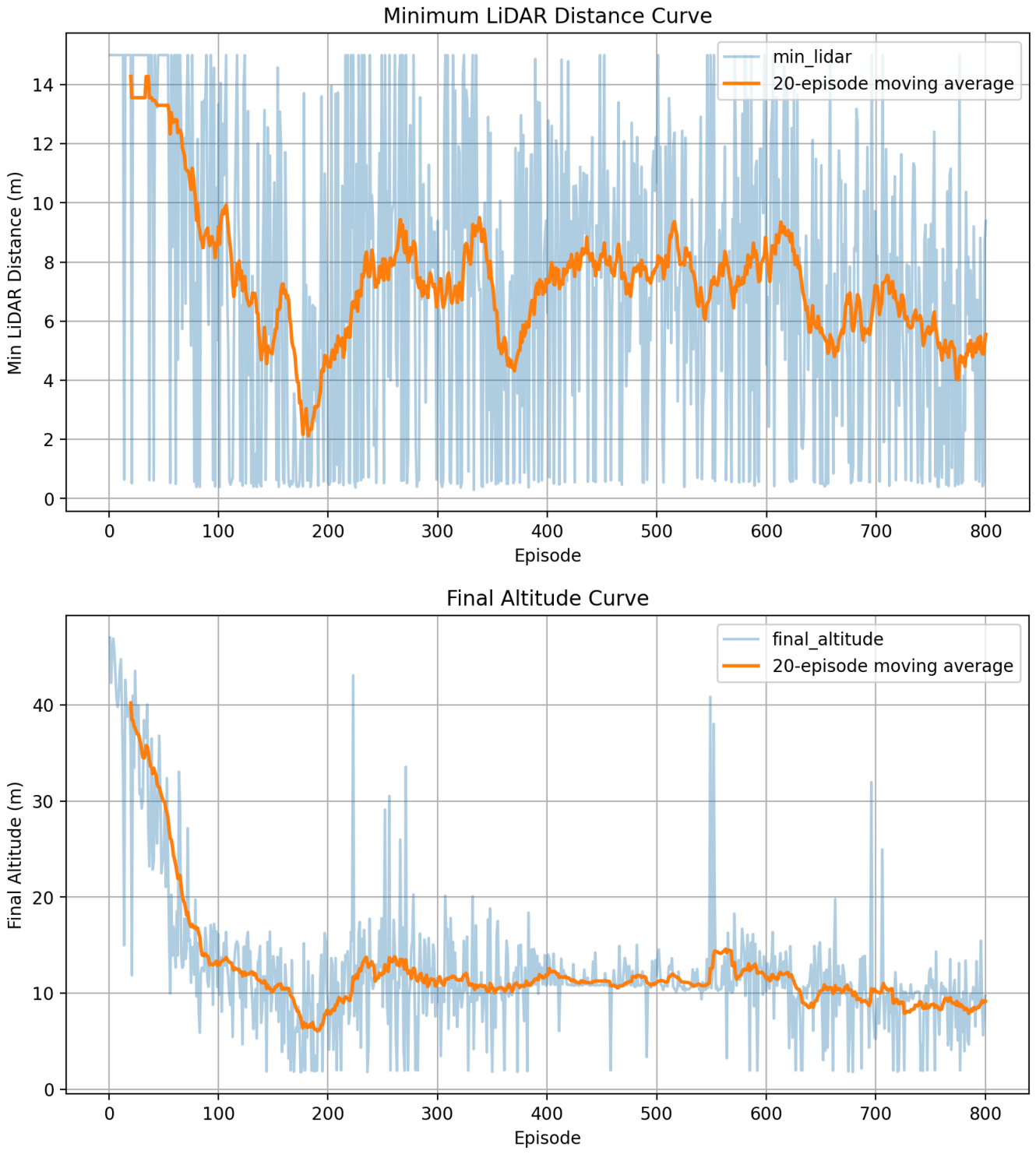


Figure 12: Safety and altitude metrics under the soft-boundary strategy.

These metrics indicate that the agent gradually learns:

- safer obstacle margins,
- more stable altitude control,
- improved environmental awareness.

The hard-boundary formulation produces more consistent altitude regulation across episodes.

4.4 Trajectory-based Policy Behavior Analysis

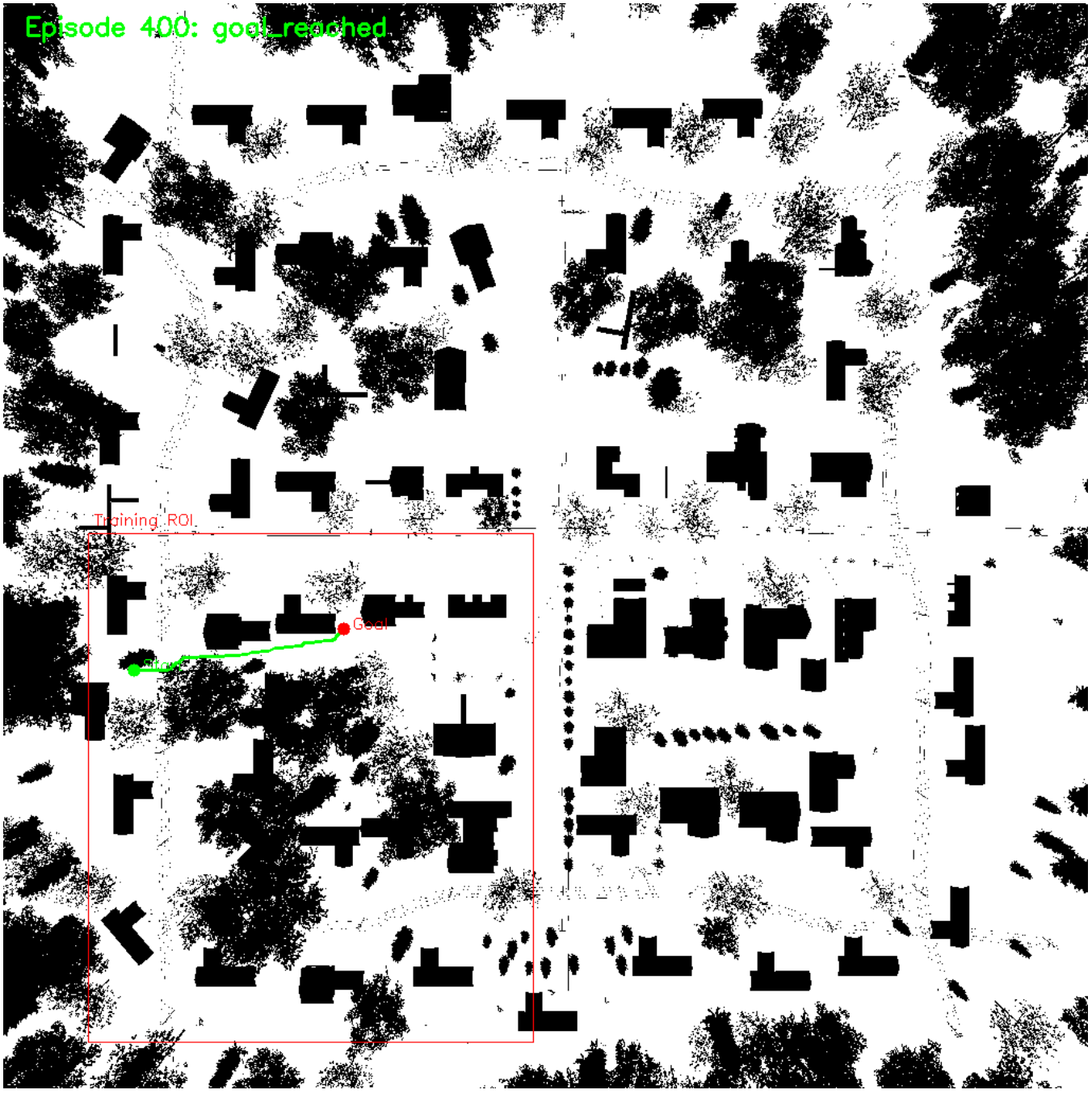
Trajectory analysis provides direct insight into the learned navigation policy and failure modes.

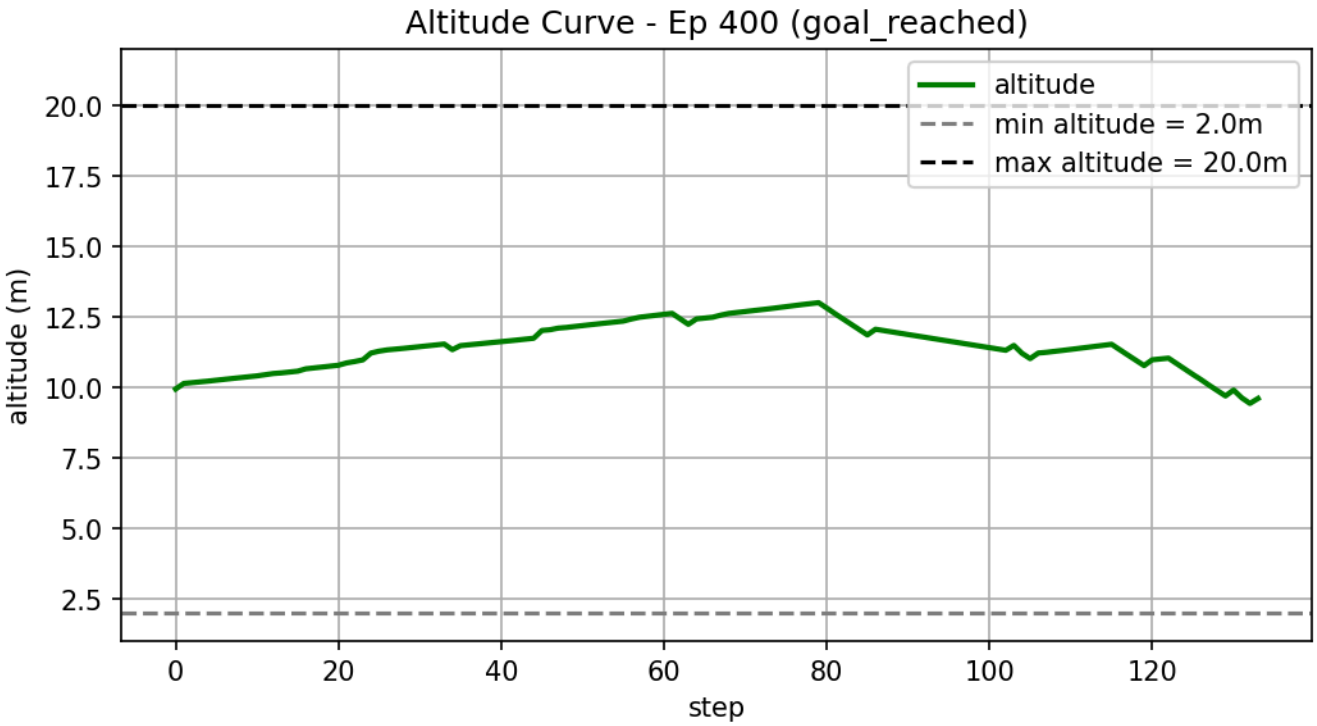
Unlike scalar training metrics alone, trajectory visualization reveals how the agent spatially reasons about:

- obstacle avoidance,
- goal pursuit,
- altitude control,
- exploration behavior.

4.4.1 Successful Navigation Policy

Hard Reward Successful Example




Altitude Curve - Ep 400 (goal_reached)
altitude
min altitude = 2.0m
max altitude = 20.0m
altitude (m)
20.0
17.5
15.0
12.5
10.0
7.5
5.0
2.5
0
20
40
60
80
100
120
step

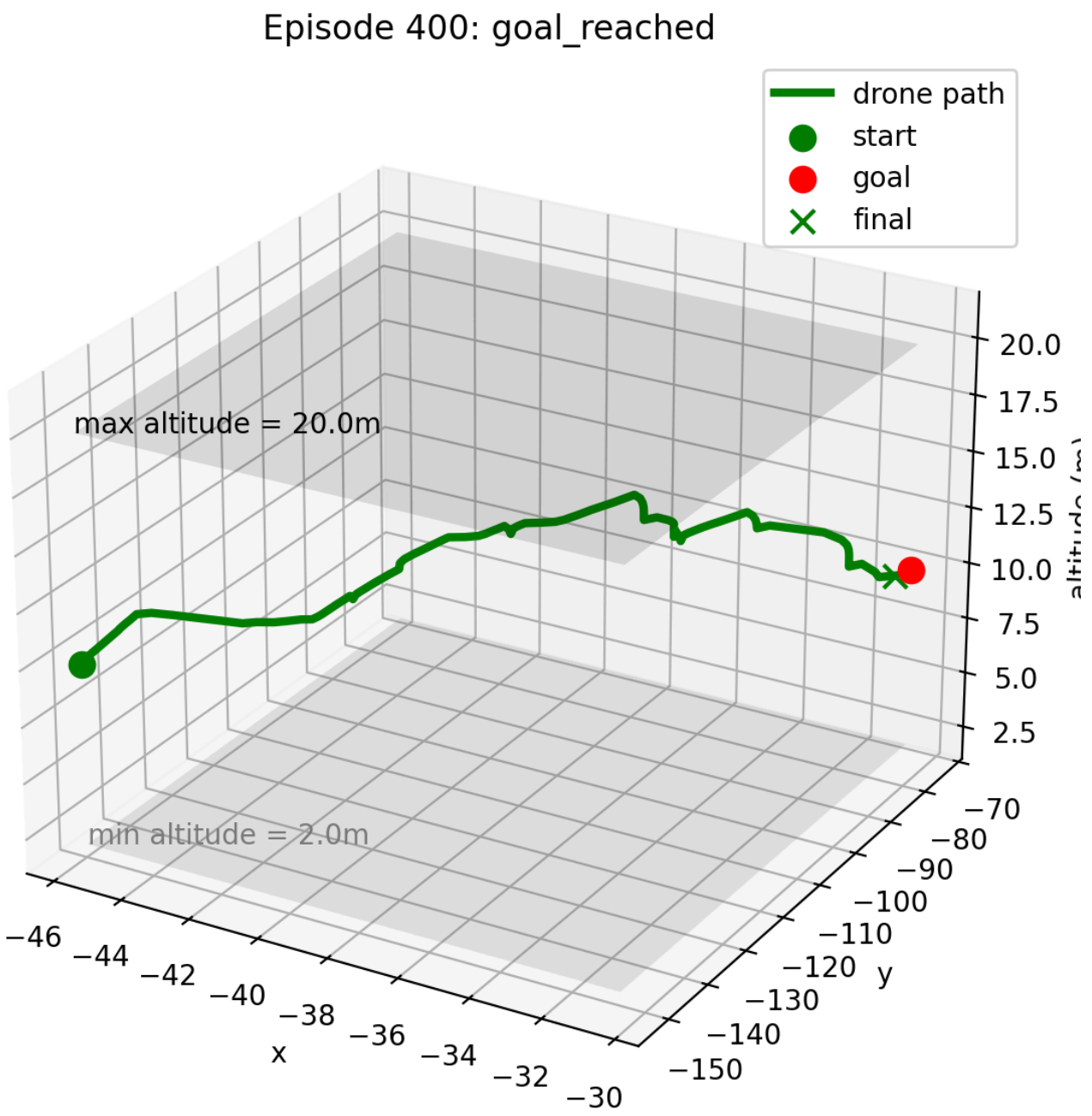


Figure 13: Successful hard-boundary trajectory.

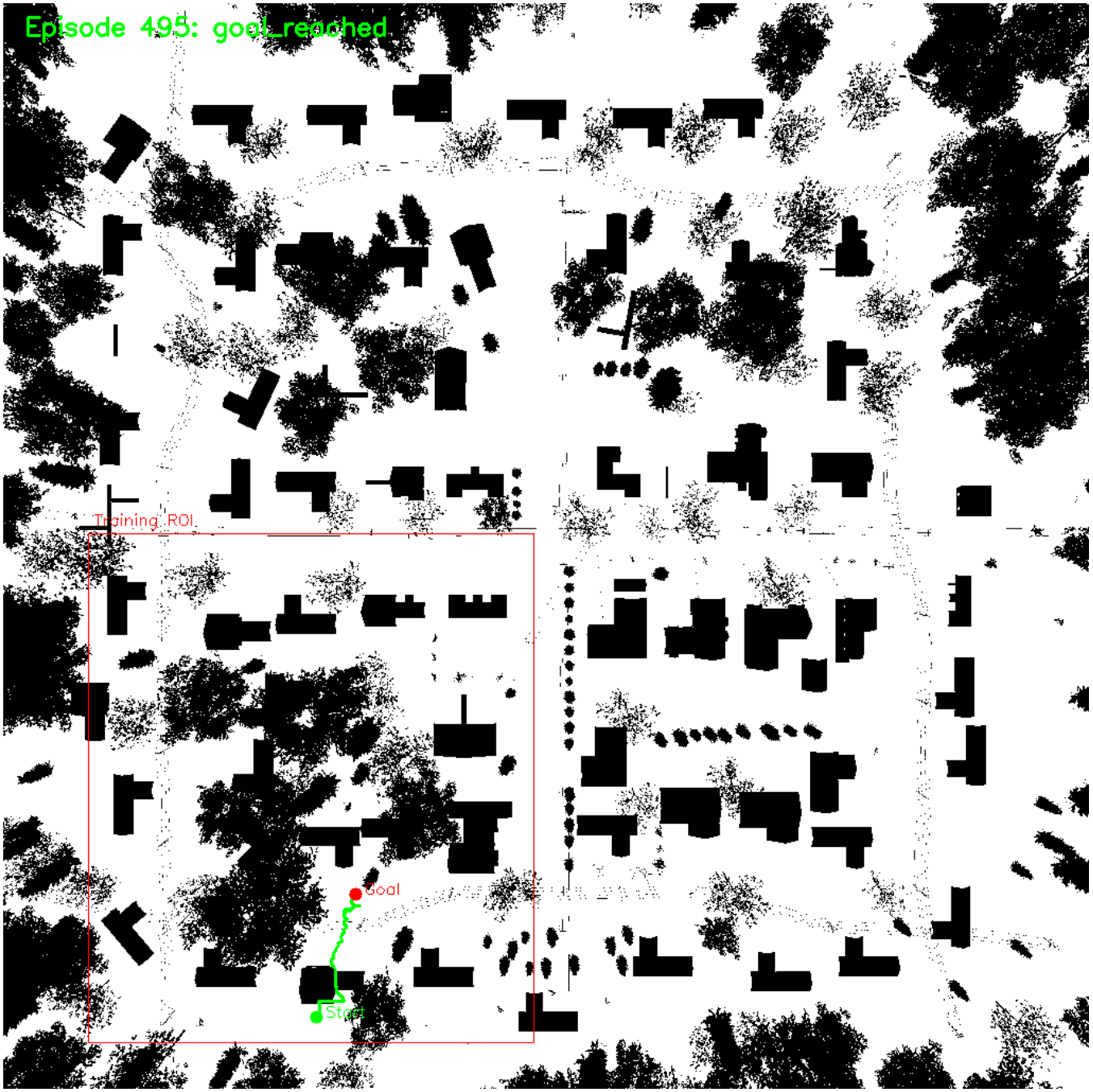
The hard-boundary policy demonstrates highly task-oriented navigation behavior. The drone maintains a relatively smooth trajectory while avoiding obstacles and remaining inside the training ROI.

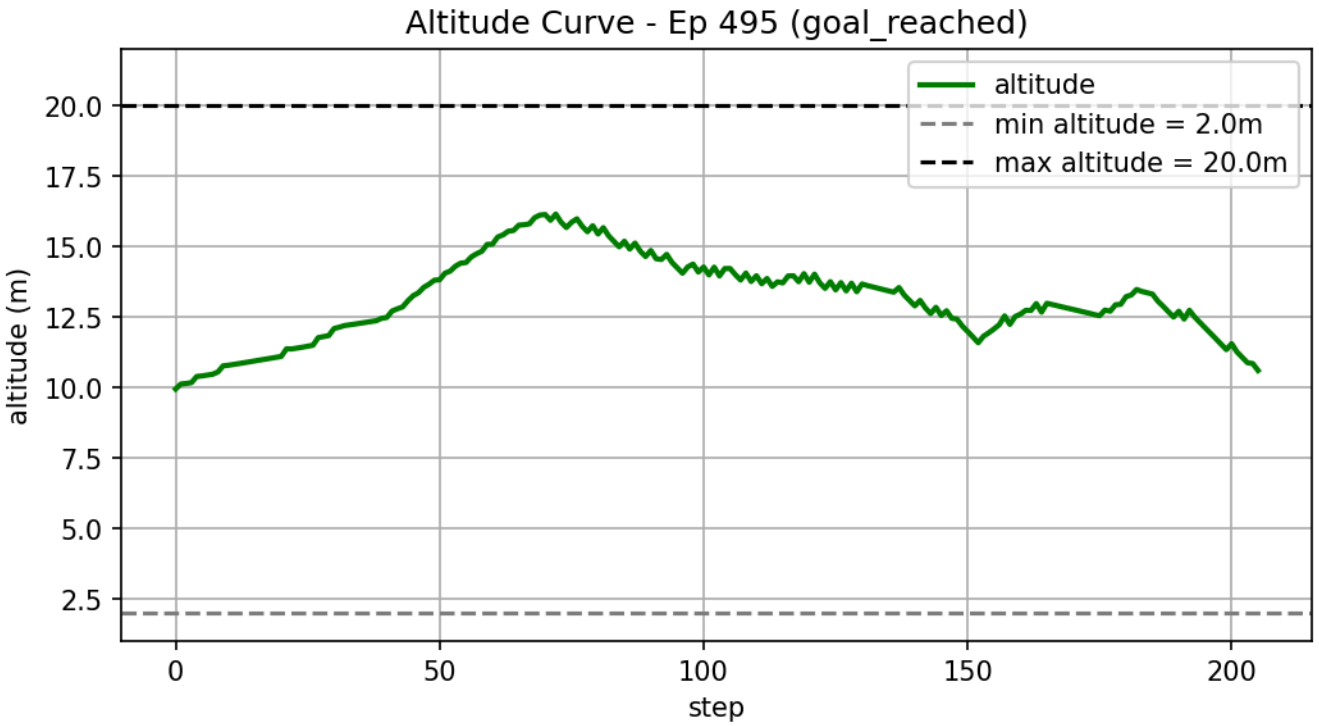
The altitude curve also shows stable vertical regulation throughout the episode. Instead of excessive oscillation, the policy learns controlled altitude adjustments that support obstacle avoidance without violating altitude constraints.

The 3D trajectory visualization further confirms that the learned policy successfully integrates:

- directional planning,
- local obstacle avoidance,
- altitude stabilization.

Soft Reward Successful Example




Altitude Curve - Ep 495 (goal_reached)
altitude
min altitude = 2.0m
max altitude = 20.0m
altitude (m)
20.0
17.5
15.0
12.5
10.0
7.5
5.0
2.5
0
50
100
150
200
step

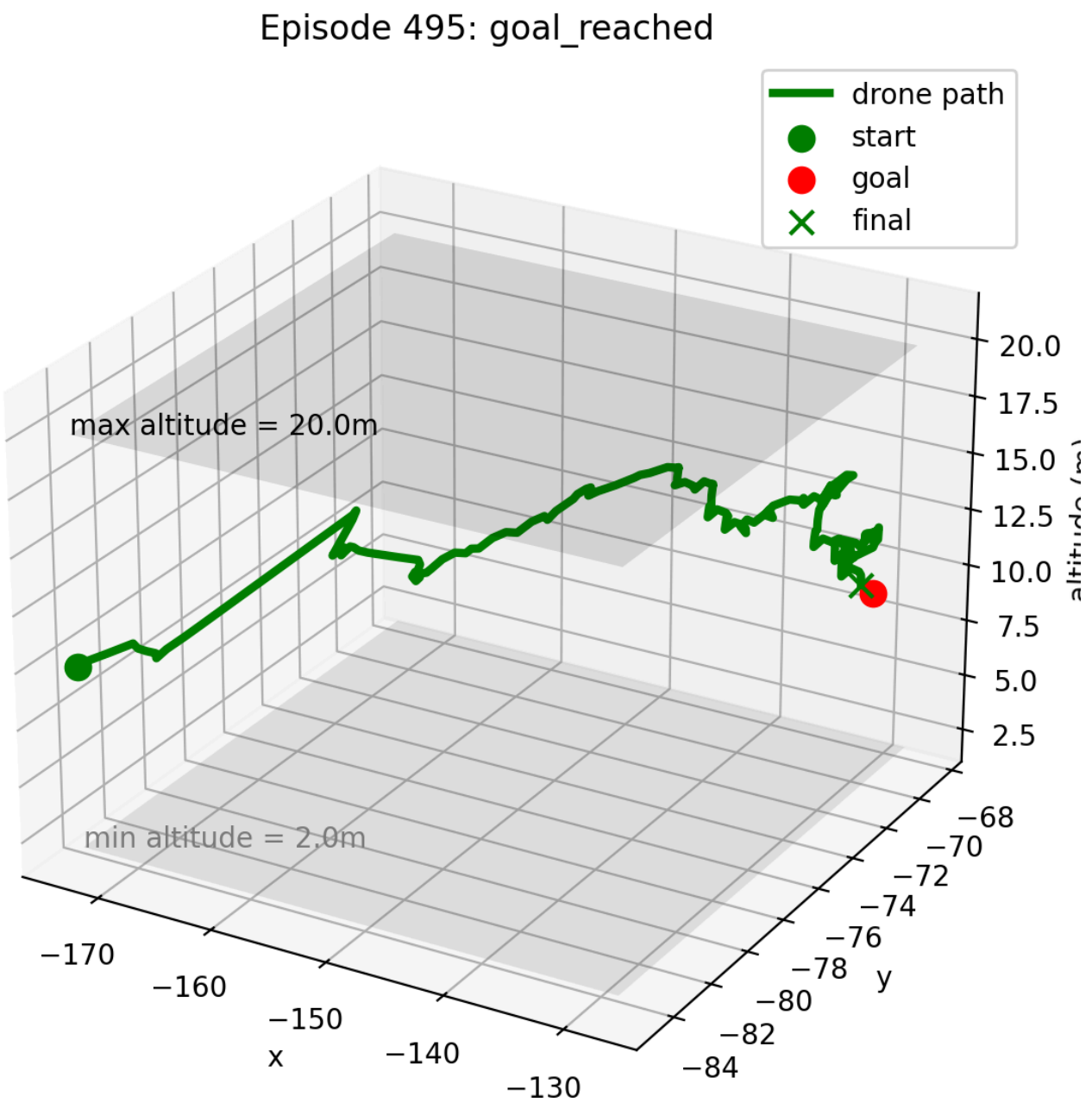


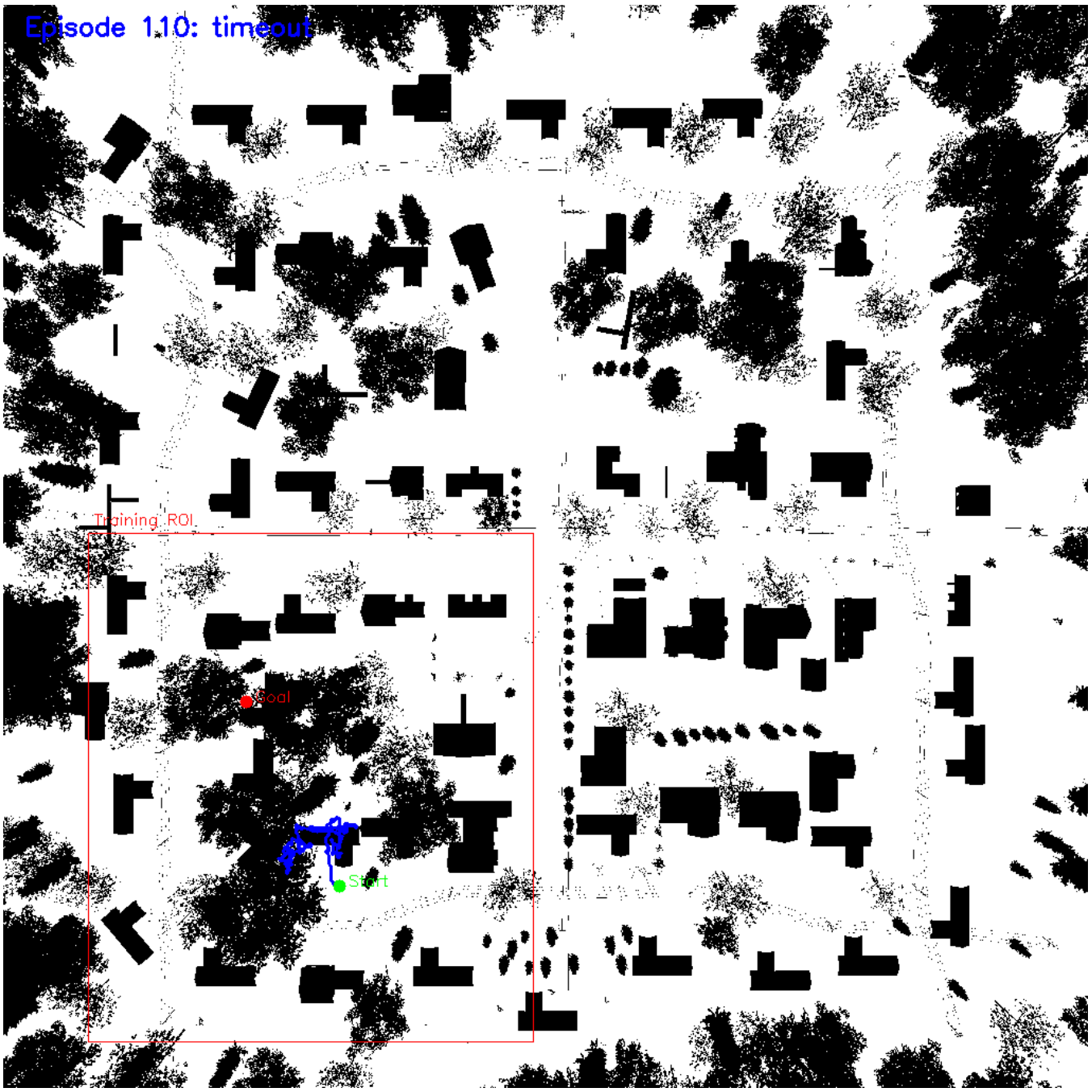
Figure 14: Successful soft-boundary trajectory.

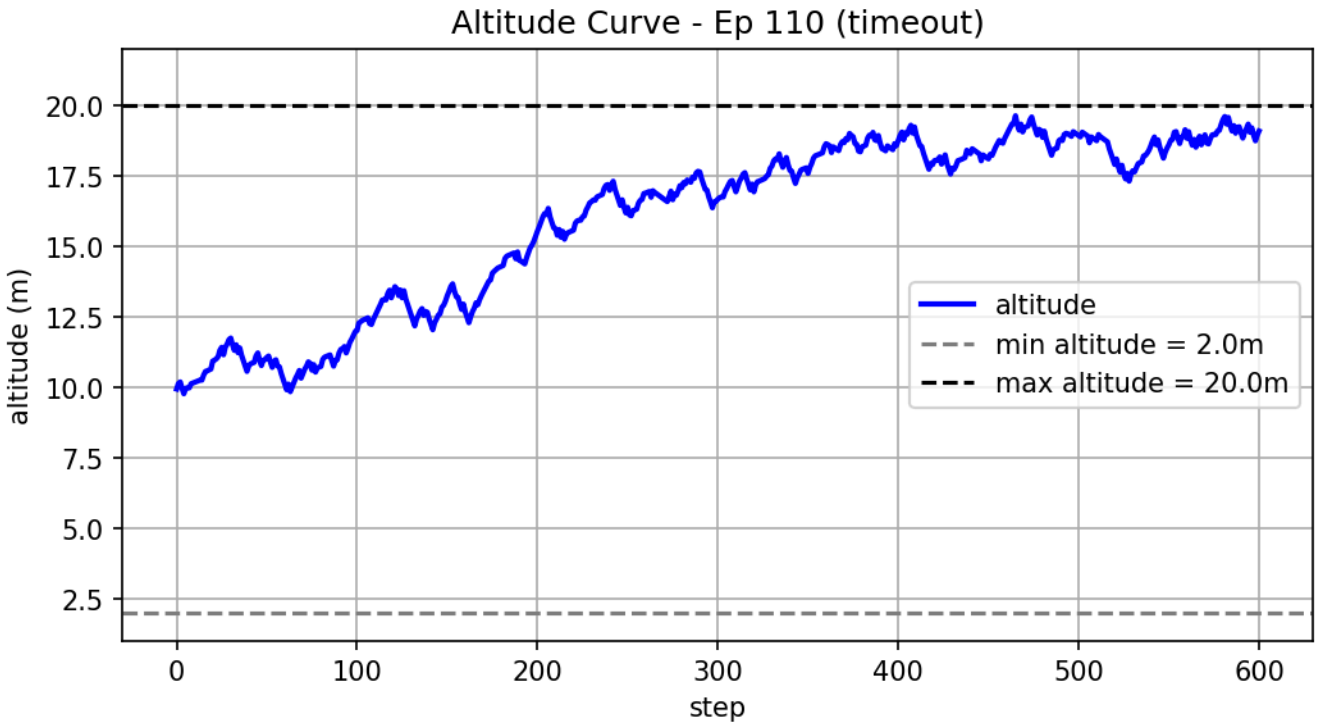
Although the soft-boundary policy eventually reaches the goal, its trajectory contains noticeably more exploratory motion and altitude fluctuation.

Compared with the hard-boundary system, the navigation behavior is less direct and less spatially efficient.

4.4.2 Timeout Failure Analysis

Hard Reward Timeout Example


Episode 110: timeout
Training ROI
Goal
Start


Altitude Curve - Ep 110 (timeout)
altitude (m)
20.0
17.5
15.0
12.5
10.0
7.5
5.0
2.5
0
100
200
300
400
500
600
step
altitude
min altitude = 2.0m
max altitude = 20.0m

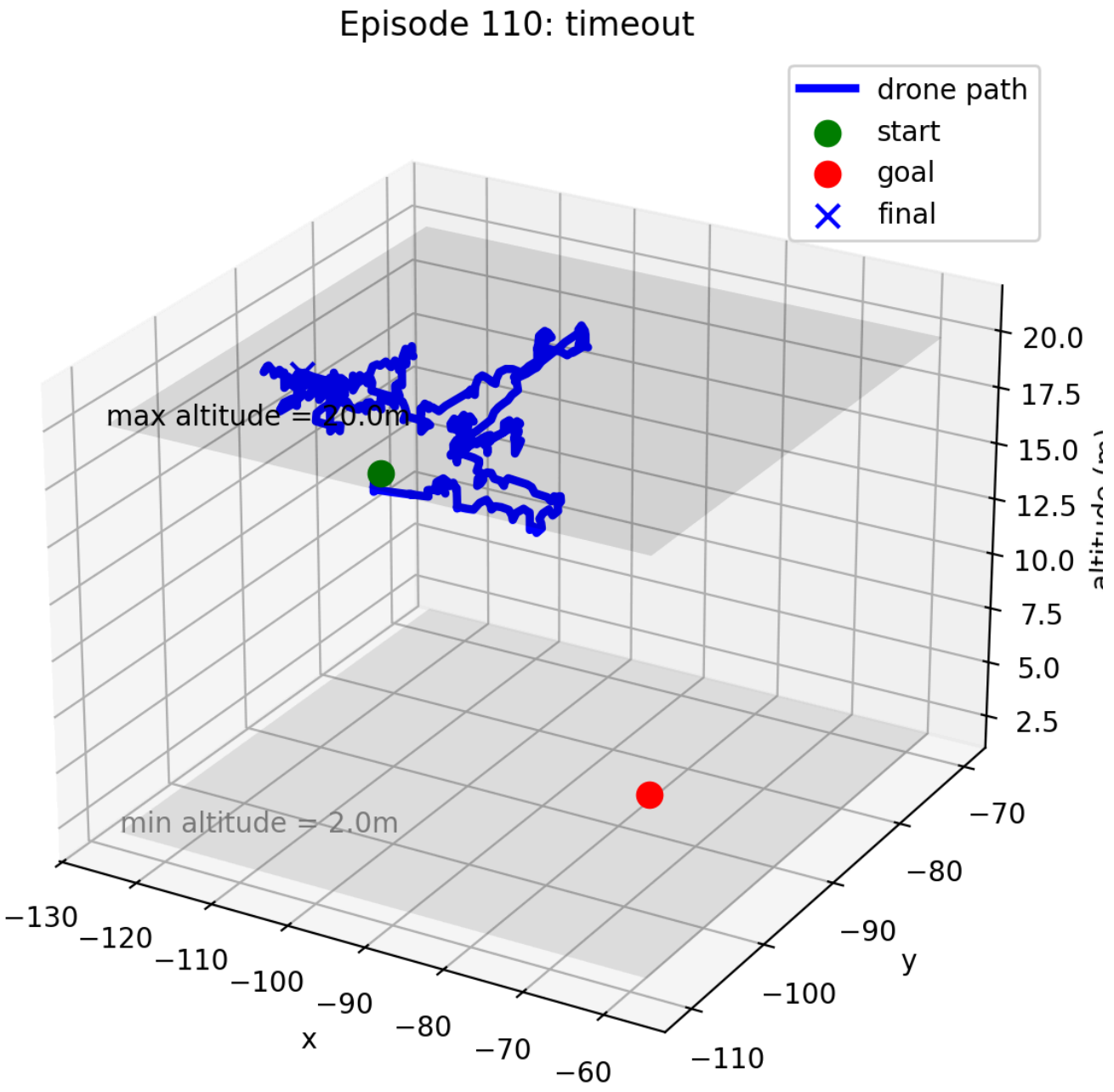
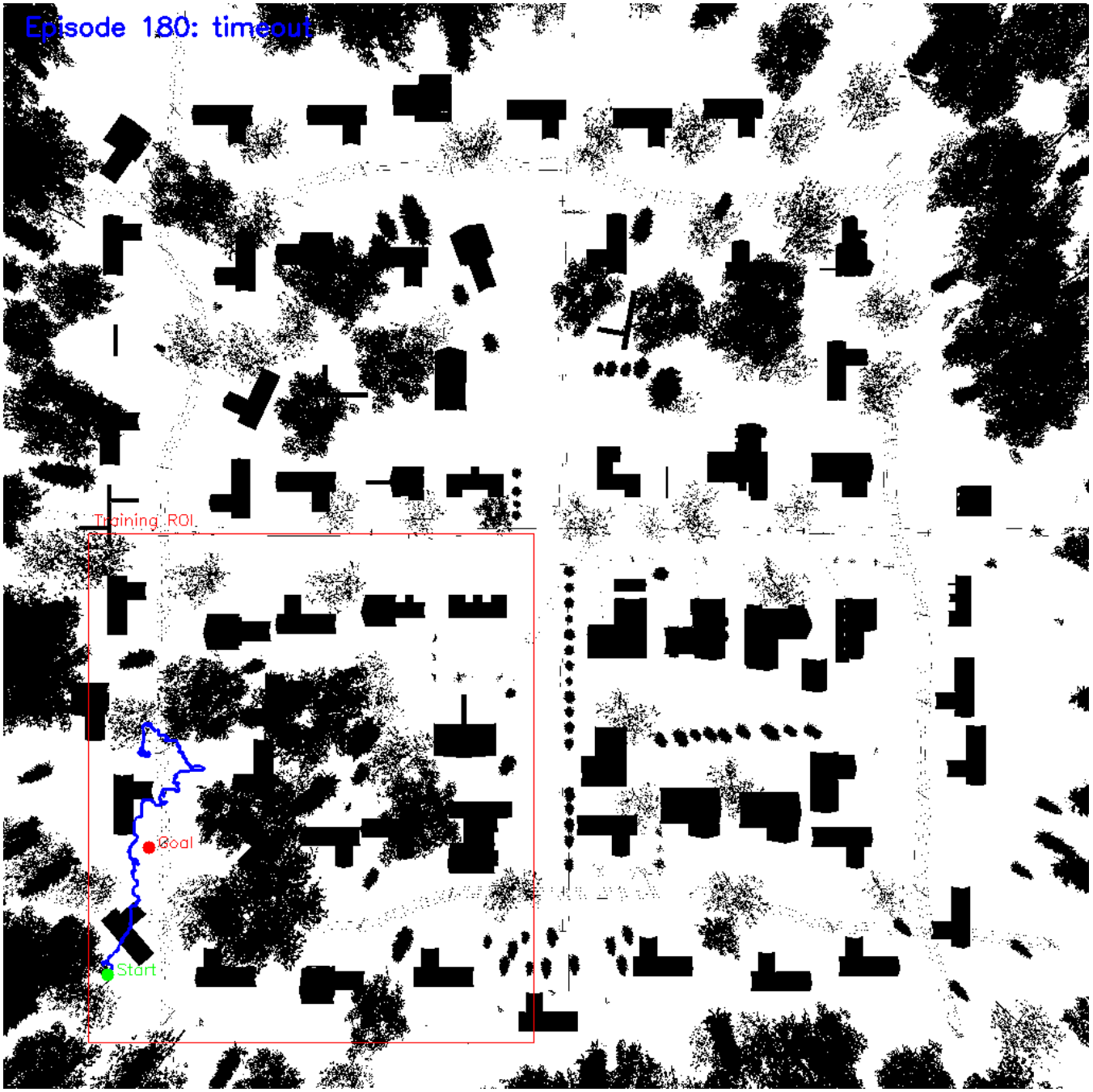


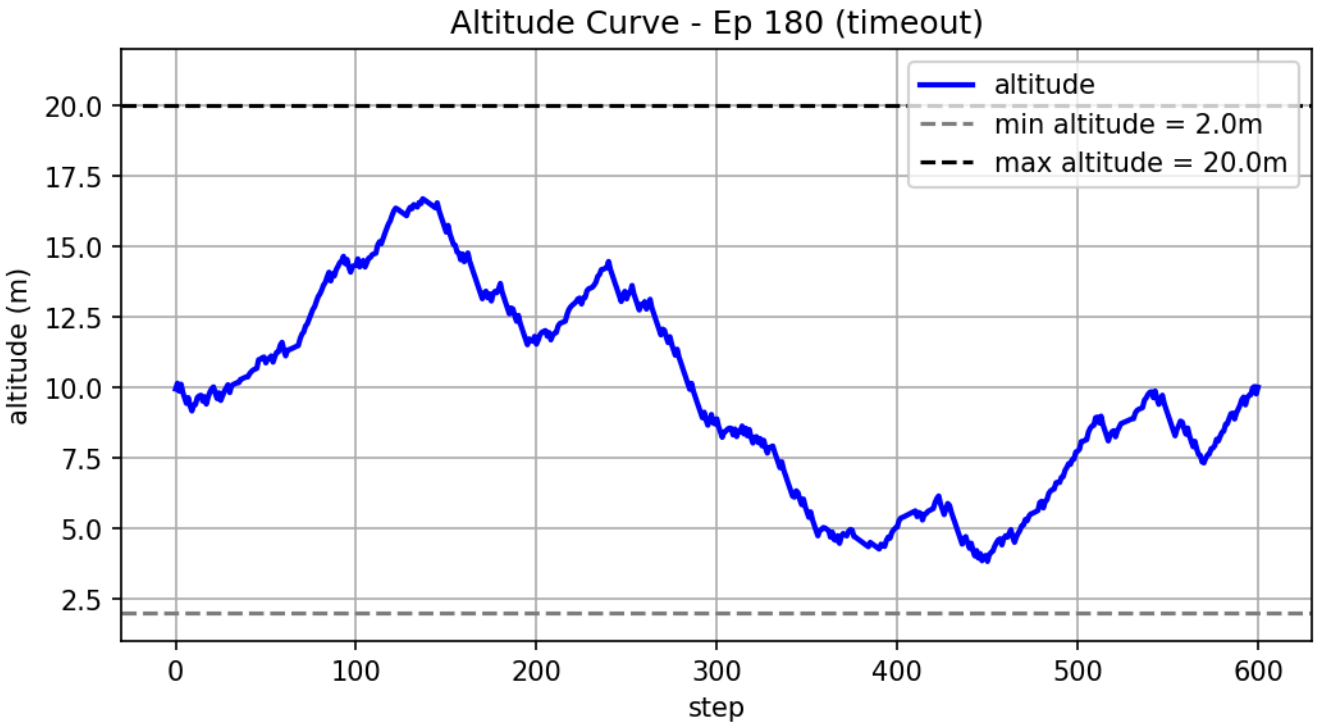
Figure 15: Timeout case under the hard-boundary strategy.

The hard-boundary timeout failure reveals that the policy occasionally becomes trapped in locally safe but globally ineffective movement patterns.

The drone avoids collisions successfully but fails to generate sufficient directional progress toward the target before the episode limit is reached.

Soft Reward Timeout Example




Altitude Curve - Ep 180 (timeout)
altitude
min altitude = 2.0m
max altitude = 20.0m
altitude (m)
20.0
17.5
15.0
12.5
10.0
7.5
5.0
2.5
0
100
200
300
400
500
600
step

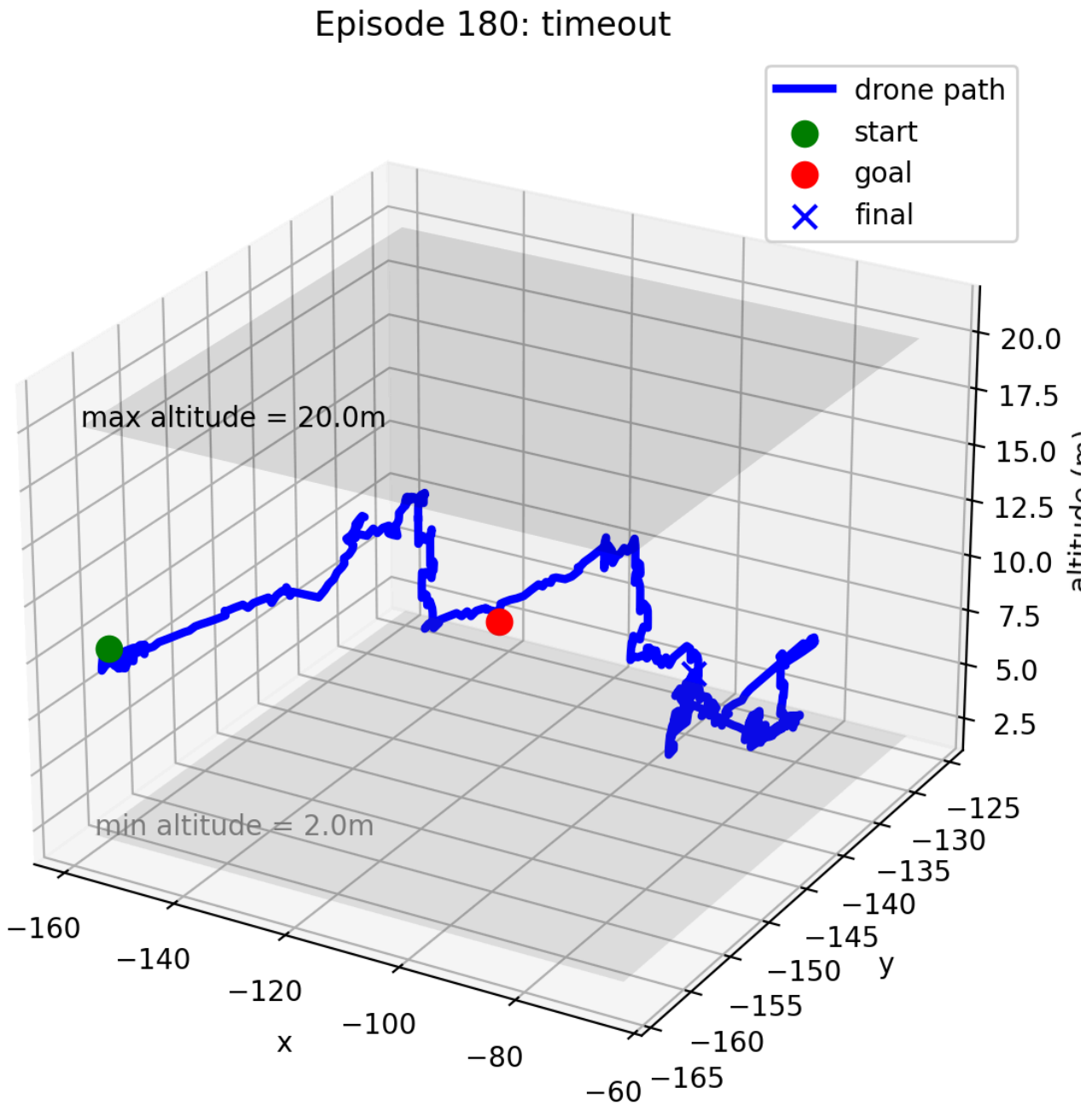


Figure 16: Timeout case under the soft-boundary strategy.

The soft-boundary timeout trajectory exhibits substantially more wandering behavior.

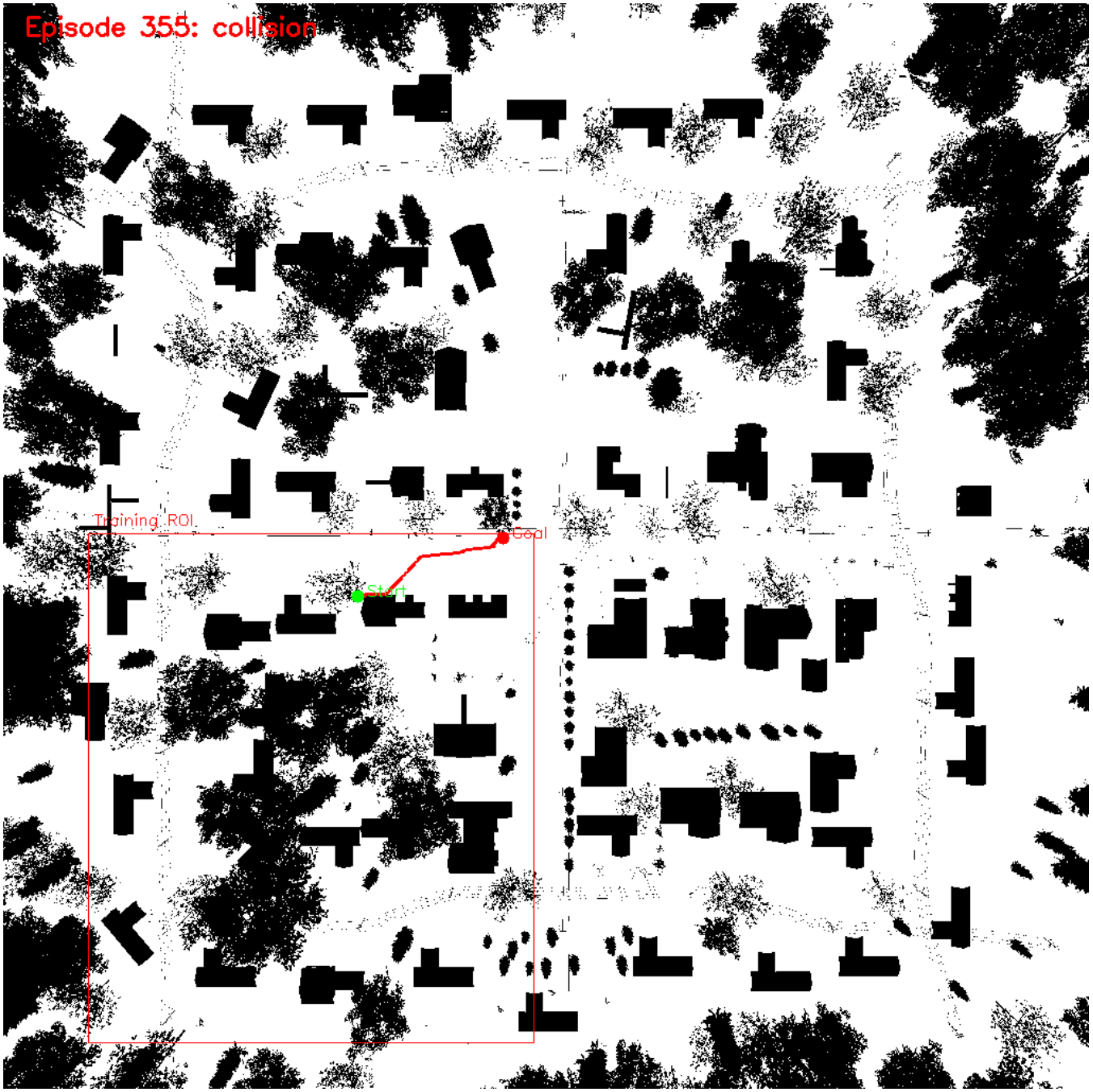
The drone repeatedly explores large regions of the environment without establishing stable directional movement toward the goal.

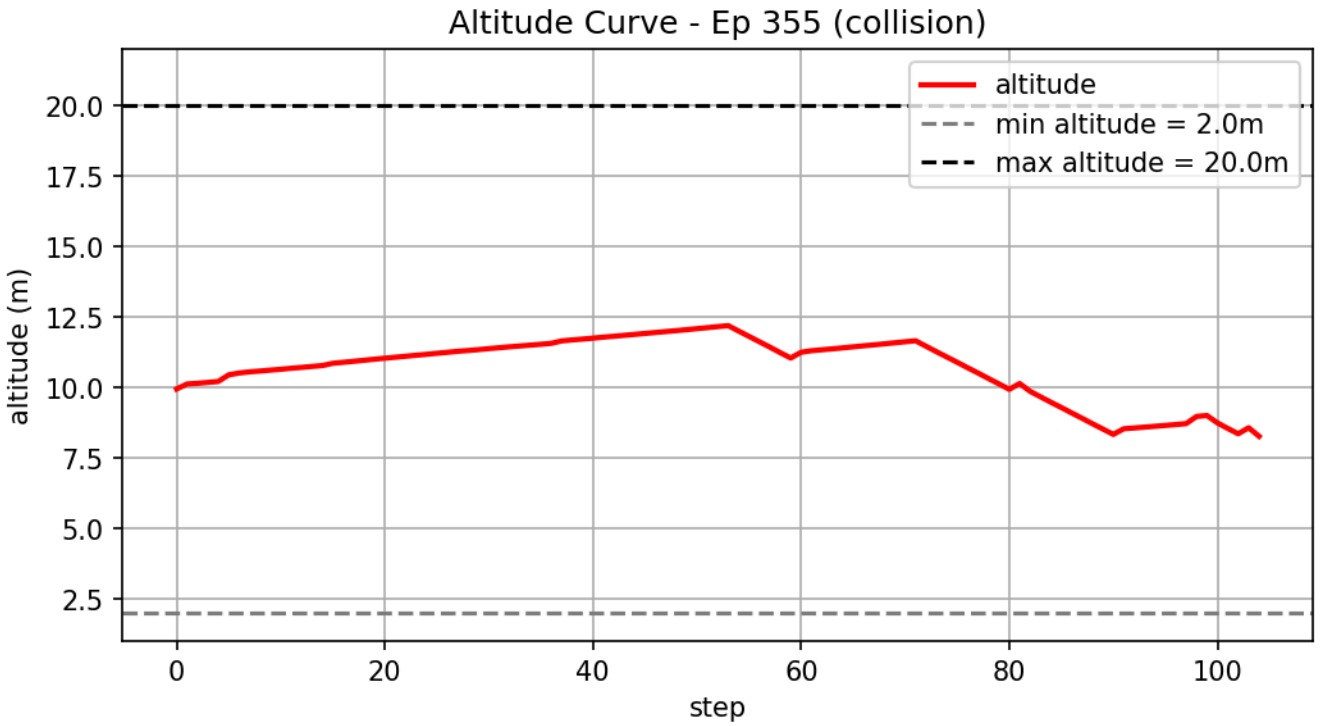
The altitude curve also shows significantly larger vertical oscillations, indicating unstable policy behavior during long exploration phases.

This suggests that soft penalties alone may be insufficient to constrain exploration effectively in large-scale sparse-goal environments.

4.4.3 Collision Failure Analysis

Hard Reward Collision Example




Altitude Curve - Ep 355 (collision)
altitude
min altitude = 2.0m
max altitude = 20.0m
altitude (m)
20.0
17.5
15.0
12.5
10.0
7.5
5.0
2.5
0
20
40
60
80
100
step

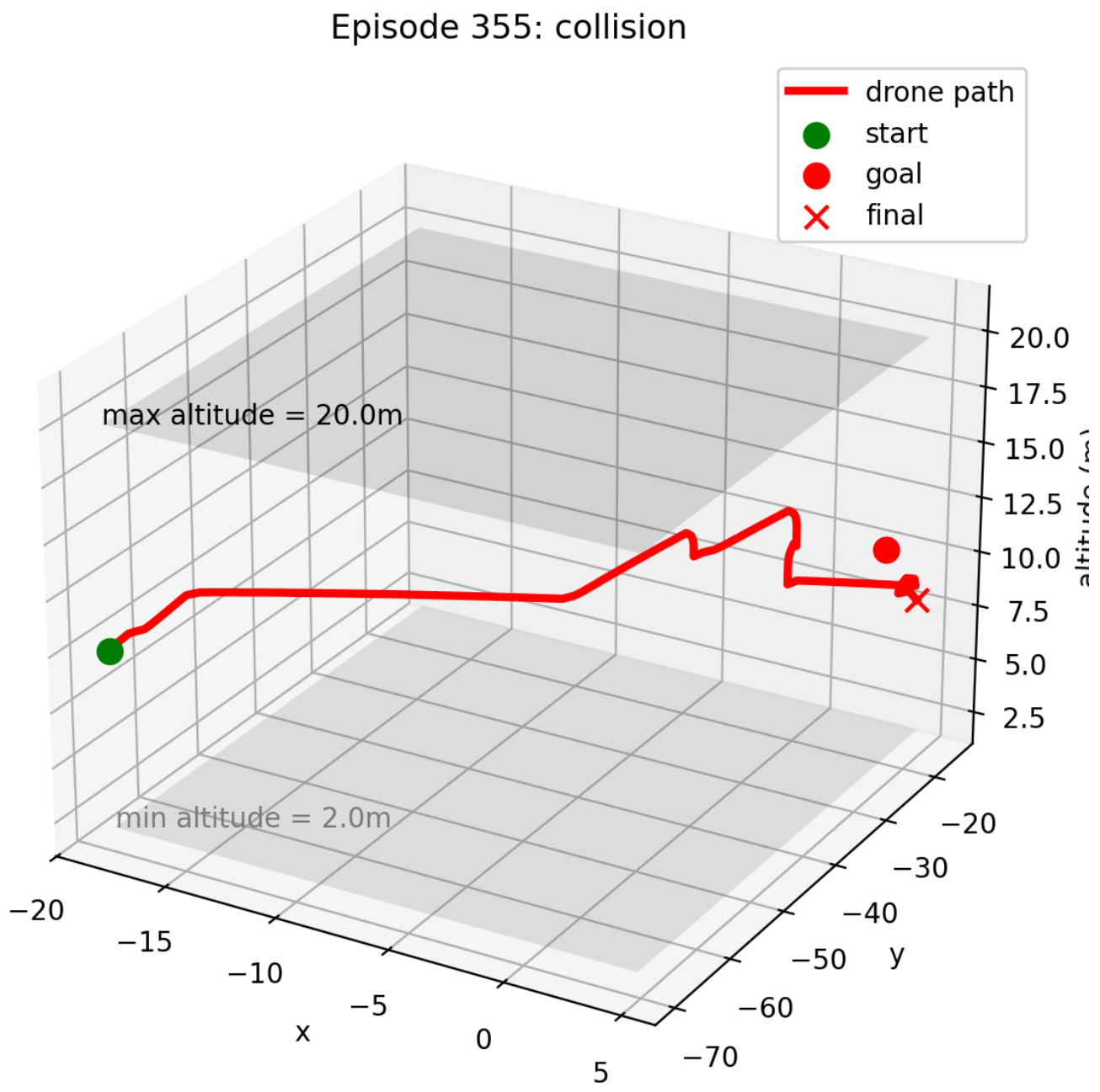


Figure 17: Collision case under the hard-boundary strategy.

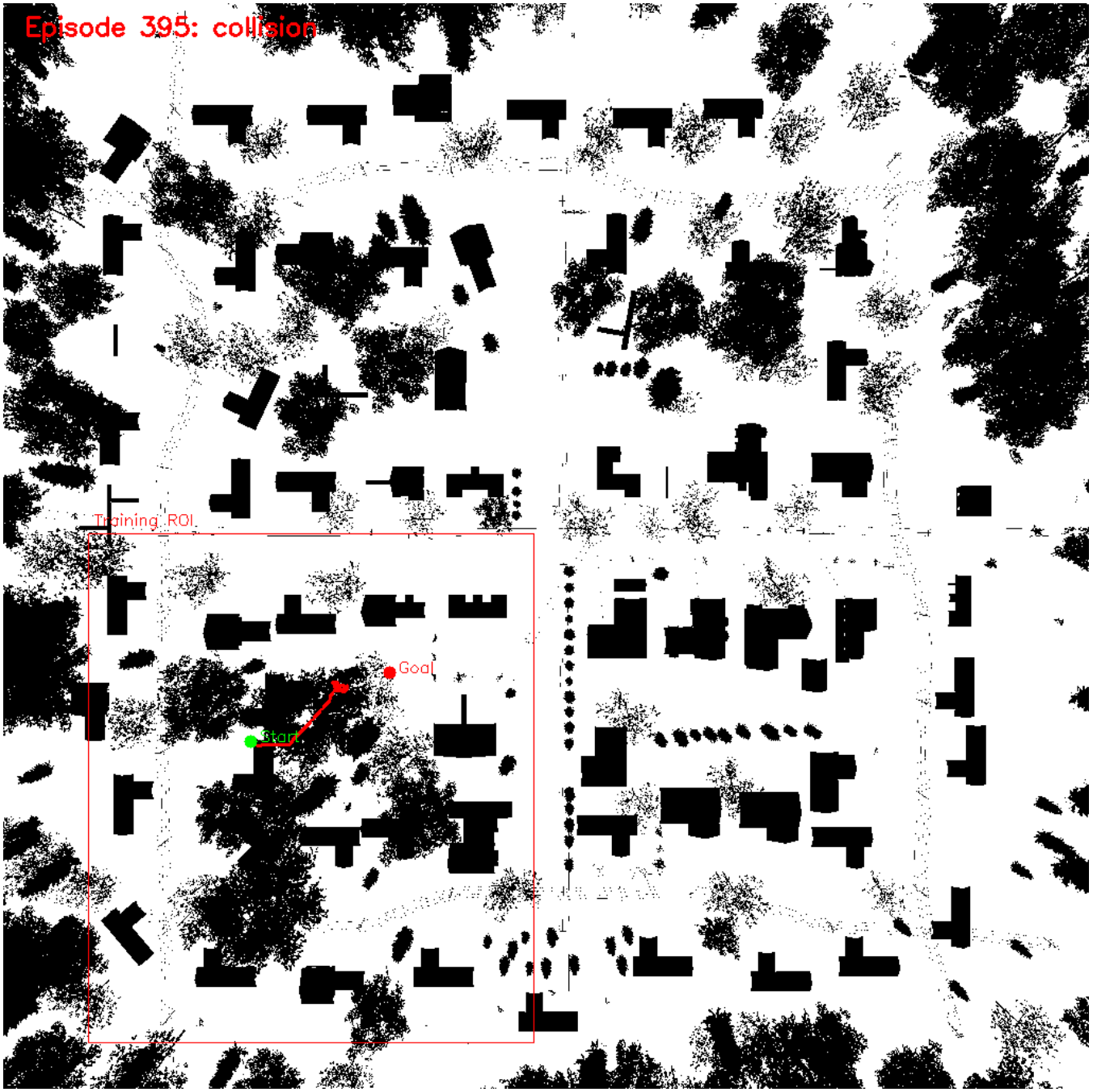
The hard-boundary collision trajectory shows that the drone successfully performs long-range navigation but eventually fails during local obstacle negotiation near the target region.

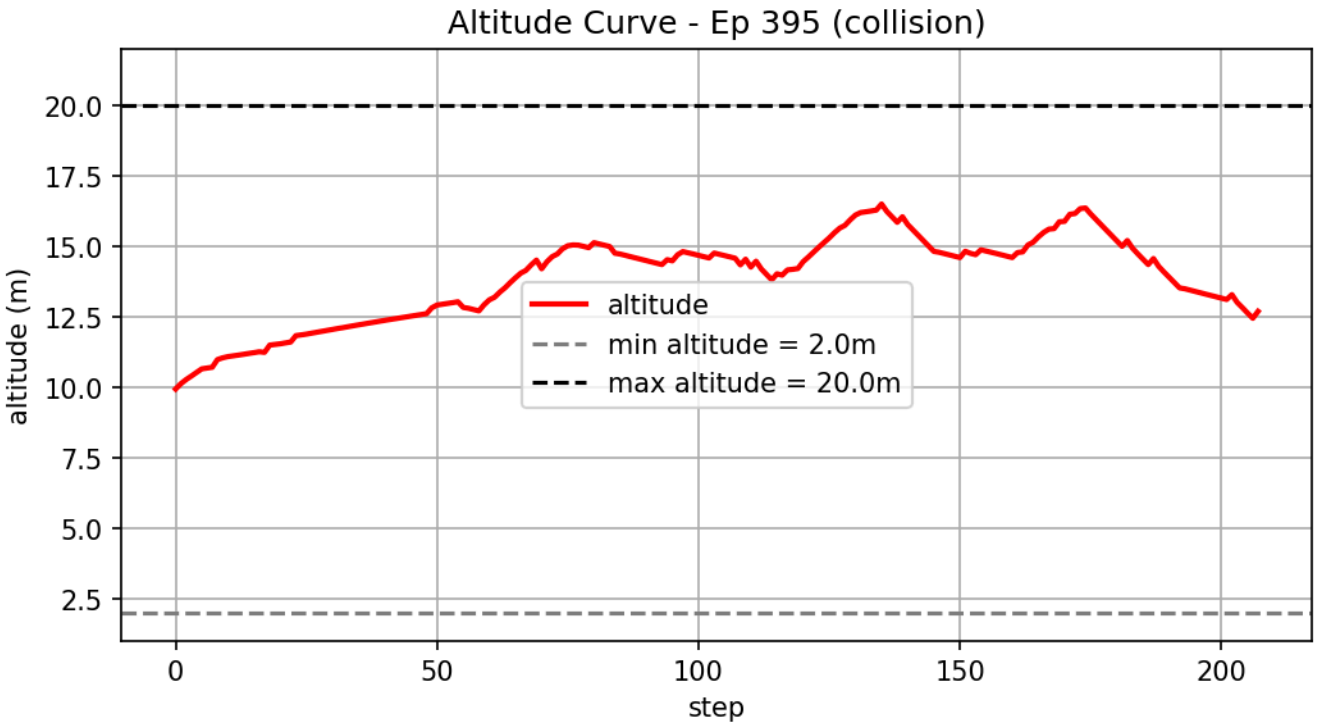
This indicates that the policy already learned:

- global directional planning,
- stable altitude control,
- long-distance navigation.

However, fine-grained obstacle interaction near dense structures remains challenging.

Soft Reward Collision Example




Altitude Curve - Ep 395 (collision)
altitude (m)
20.0
17.5
15.0
12.5
10.0
7.5
5.0
2.5
0
50
100
150
200
step
altitude
min altitude = 2.0m
max altitude = 20.0m

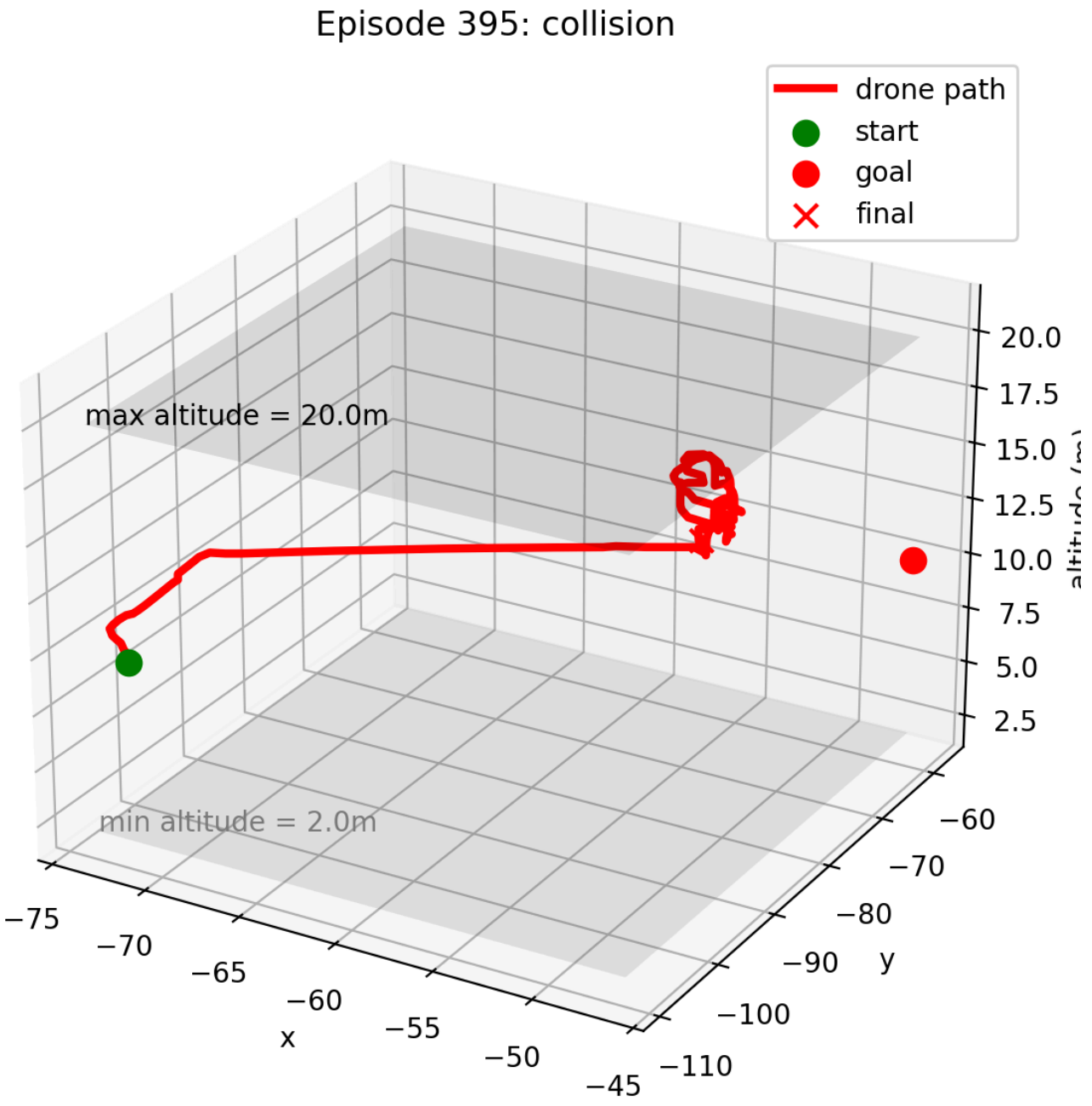


Figure 18: Collision case under the soft-boundary strategy.

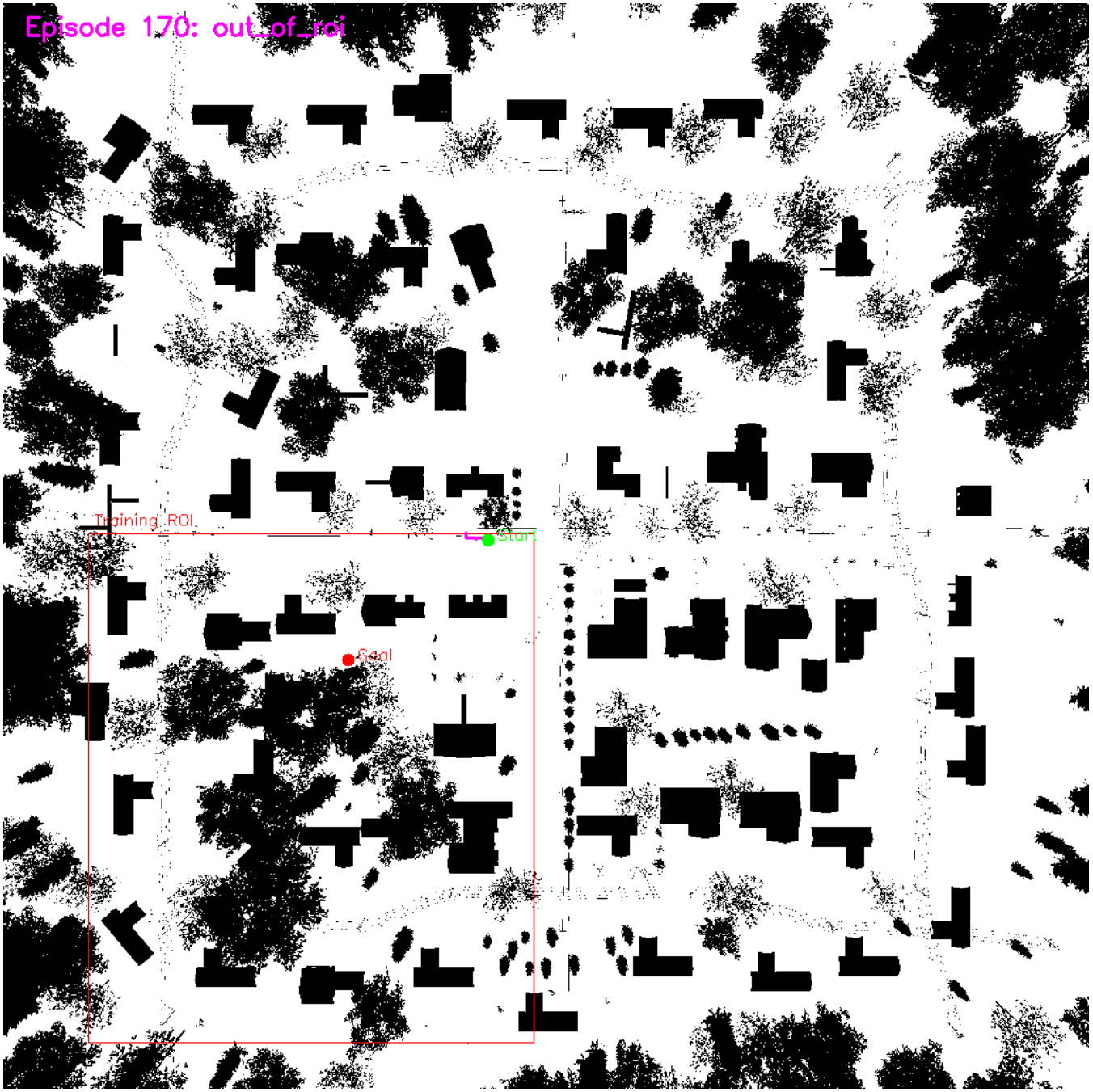
The soft-boundary collision example demonstrates excessive local oscillation near obstacles.

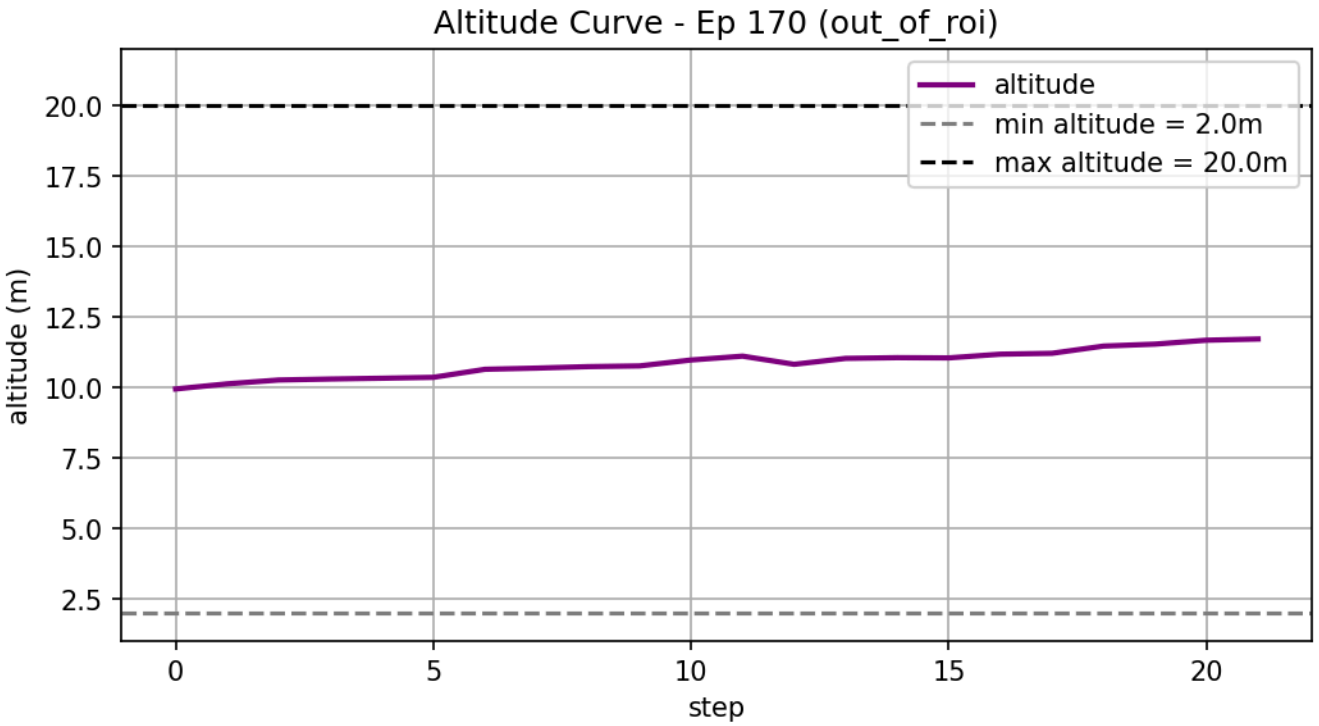
Instead of committing to a stable avoidance maneuver, the drone repeatedly circles within a constrained region before eventually colliding.

This behavior suggests that overly permissive exploration may reduce local decision consistency during obstacle avoidance.

4.4.4 ROI Failure Analysis

Hard Reward ROI Failure Example




Altitude Curve - Ep 170 (out_of_roi)
altitude
min altitude = 2.0m
max altitude = 20.0m
altitude (m)
20.0
17.5
15.0
12.5
10.0
7.5
5.0
2.5
0
5
10
15
20
step

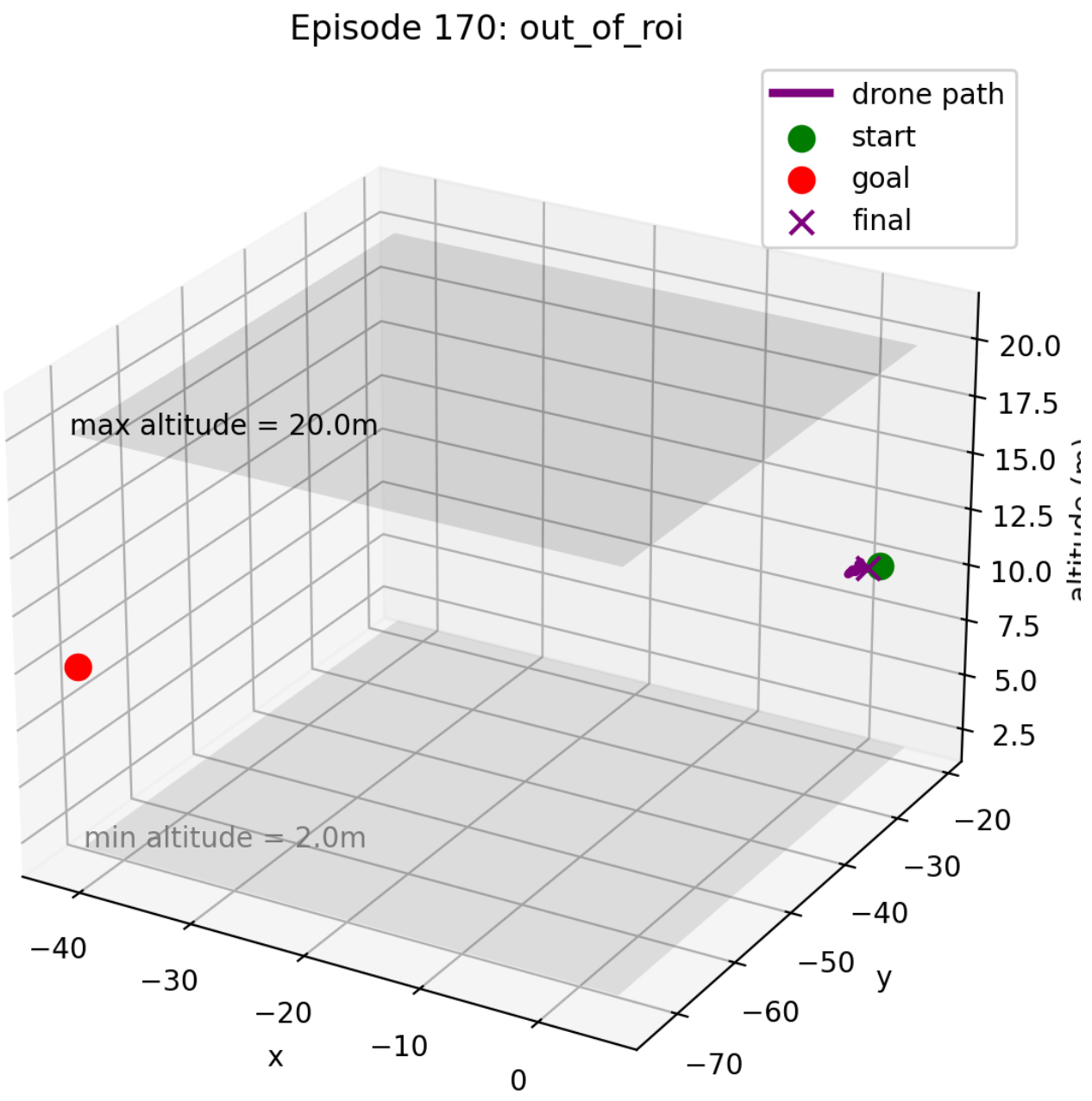


Figure 19: ROI violation case under the hard-boundary strategy.

The ROI failure trajectory demonstrates how early-stage exploration occasionally drives the drone outside the intended operational region.

However, because the hard-boundary system immediately terminates these episodes, the policy quickly learns to avoid unproductive exploration patterns during later training stages.

The soft-boundary system does not contain comparable ROI termination examples because out-of-bound exploration does not immediately reset the environment.

4.5 Discussion

The experiments demonstrate that reward engineering plays a critical role in reinforcement learning-based autonomous navigation.

Several important observations emerge from the results:

1. Dense reward shaping significantly improves learning efficiency in sparse-goal environments.
2. Hierarchical obstacle penalties encourage proactive safety behavior rather than reactive collision avoidance alone.
3. Hard environmental constraints improve convergence speed and reduce unstable exploration.
4. Trajectory-based analysis provides substantially deeper insight into learned policy behavior than scalar metrics alone.

Although the proposed DQN framework successfully learns stable navigation policies, several limitations remain:

- local obstacle negotiation near dense structures,
- exploration efficiency in large environments,
- discrete-action maneuver smoothness.

Future work may explore:

- PPO or SAC continuous control,
- dynamic obstacle environments,
- memory-enhanced policies,
- transformer-based world models,
- multi-agent coordination systems.

5. System Integration, Limitations, and Future Work

5.1 Integrated System Perspective

- A* provides global structure.
- APF provides reactive local avoidance.
- FSM recovery handles APF local-minimum/deadlock cases.
- RL explores a learned alternative for local 3D navigation.
- The project should be presented as a comparison and integration of classical and learning-based navigation, not as RL replacing A*/APF.

5.2 Comparison Between Classical and RL Navigation

- A*: deterministic, interpretable, efficient, but static-map dependent.
- APF/FSM: reactive and lightweight, but sensitive to local minima and parameter tuning.
- DQN: adaptive and data-driven, but expensive to train and sensitive to reward design.
- Best practical system: A* for global waypoint planning + RL/APF for local control.

5.3 Current Limitations

- Single-map training/evaluation.
- Static obstacle environment.
- DQN discrete action space causes less smooth motion.

- Reward design remains hand-engineered.
- RL generalization to new maps is not yet proven.
- Dense trees/obstacles still cause collision or timeout cases.

5.4 Future Work

- Continuous-control RL: PPO/SAC.
- Curriculum learning: 10m → 6m → 3m altitude.
- Domain randomization across maps.
- Dynamic obstacles.
- Better LiDAR representation, possibly CNN/Transformer over point-grid observations.
- Hybrid planner: A* generates subgoals, RL handles local movement.